

UNIVERSITATEA TEHNICĂ „GHEORGHE ASACHI” DIN IAȘI

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

DOMENIUL: INGINERIA SISTEMELOR

SPECIALIZAREA: AUTOMATICĂ ȘI INFORMATICĂ APLICATĂ

PROIECT DE DIPLOMĂ

Sistem încorporat de securitate și control al accesului, cu comunicație prin Internet și aplicație mobilă

Coordonator științific

Conf. Dr. Ing. Mihai Postolache

Absolvent

Alexandru Agape

Iași, 2018

DECLARAȚIE DE ASUMARE A AUTENTICITĂȚII
LUCRĂRII DE LICENȚĂ

Subsemnatul(a) _____,
legitimat(a) cu _____ seria _____ nr. _____ , CNP _____
autorul lucrării _____

elaborată în vederea susținerii examenului de finalizare a studiilor de licență organizat de către Facultatea de Automatică și Calculatoare din cadrul Universității Tehnice „Gheorghe Asachi” din Iași, sesiunea _____ a anului universitar _____, luând în considerare conținutul Art. 34 din Codul de etică universitară al Universității Tehnice „Gheorghe Asachi” din Iași (Manualul Procedurilor, UTI.POM.02 – Funcționarea Comisiei de etică universitară), declar pe proprie răspundere, că această lucrare este rezultatul propriei activități intelectuale, nu conține porțiuni plagate, iar sursele bibliografice au fost folosite cu respectarea legislației române (legea 8/1996) și a convențiilor internaționale privind drepturile de autor.

Data

Semnătura

Cuprins

1. Introducere	1
2. Resurse hardware și software.....	3
2.1. Placa mini PC Raspberry Pi	3
2.2. Camera de luat vederi	4
2.3. Încuietoare electromagnetă și circuit de alimentare	5
2.4. Biblioteca OpenCV	7
2.5. Biblioteca LibXML.....	8
3. Implementarea proiectului	9
3.1. Aplicația server	11
3.1.1. Împărțirea aplicației pe fire de execuție	11
3.1.2. Preluarea și procesarea imaginilor (main, detect și print).....	16
3.1.3. Firul de execuție ServerTCP	22
3.1.4. Firul de execuție serverUDP	25
3.1.5. Firul de execuție Door.....	28
3.1.6. Baza de date a utilizatorilor și desfășurător	29
3.2. Aplicația mobilă (client)	31
3.2.1. Prezentare generală	31
3.2.2. Componentele aplicației mobile.....	33
3.2.3. Metodele de proiectare folosite (<i>design patterns</i>).....	38
3.2.4. Clasa TcpClient.....	40
3.2.5. Clasa UdpClient	43
3.2.6. Listele cu utilizatori și acțiuni	45
3.2.7. Autentificarea biometrică cu amprentă	47
4. Rezultate și concluzii	50
Bibliografie	53
Anexa 1 - Cod sursă pentru aplicația server	
Anexa 2 - Cod sursă pentru aplicația mobilă	

1. Introducere

Nevoia de a securiza proprietatea și de a restricționa accesul în arii speciale ale unei clădiri a fost întotdeauna extrem de importantă pentru siguranța vieții și a bunurilor personale sau a bunurilor unei organizații. Încuietori mecanice sau electronice sunt disponibile în prezent, și sunt folosite pentru a preveni accesul persoanelor neautorizate în locuințe, laboratoare de cercetare, bănci, magazine, etc. Acestea sunt bazate pe diferite tehnologii de identificare precum folosirea tastaturilor alfanumerice, date biometrice (amprentă, recunoaștere facială, etc), dispozitive RFID (Radio Frequency Identification).

Primele case inteligente au fost idei, nu clădiri. Timp de decenii, cărțile science-fiction au explorat ideea de automatizare a clădirii. Scriitori precum Ray Bradbury, au imaginat un viitor în care clădirile sunt interactive și care se susțin singure. El descrie clădirile automatizate în lucrarea *There Will Come Soft Rain*, ca fiind capabile să funcționeze chiar după ce rasa umană ar dispărea [1]. La începutul secolului 21 au început să apară primele clădiri automatizate.

Ideea din spatele automatizării unei clădiri este aceea de a implementa sisteme de control și monitorizare pentru diferite instalații de încălzire, ventilație, iluminare sau alte servicii precum controlul accesului, efectuarea automată a curățeniei etc. Cu noile avansuri tehnologice în ceea ce privește puterea de calcul și tehnicile de comunicație, multe aplicații care în trecut solicitau sisteme încorporate care funcționau într-o manieră izolată, pot acum să fie interconectate într-un Internet al Lucrurilor (Internet of Things – IoT). Acesta reprezintă o rețea de dispozitive, vehicule, aparate electronice etc, care au integrate sisteme cu microprocesoare, senzori și aplicații software conectate la Internet, ceea ce le dă posibilitatea de a comunica autonom între ele. Aplicațiile pentru dispozitivele conectate la Internet sunt extrem de variate. Acestea au fost împărțite în mai multe categorii: aplicații industriale, business, și aplicații pentru consumatori. *George Osborne*, fost ministru de finanțe britanic, a declarat faptul că IoT este următoarea etapă în revoluția informațională.

Însă, aceste tehnologii noi, pe cât de revoluționare sunt, pe atât sunt de costisitoare. De aceea, în elaborarea acestui proiect, s-a ales implementarea unei aplicații în domeniul automatizării clădirilor, în care costurile sunt reduse.

Această lucrare prezintă un sistem de control al accesului basat pe o placă Raspberry Pi3 model B, conectată la Internet în rețeaua locală. Sistemul monitorizează în timp real ușa de la intrare prin intermediul unei camere de luat vederi și funcționalitate de detecție a fețelor. Sistemul notifică administratorul în situația în care o persoană petrece mai mult timp în fața ușii pentru o perioadă mai mare decât o limită prestabilită. Utilizatorii înregistrați au accesul autorizat prin intermediul unei aplicații de mobil, iar identificarea se face prin introducerea parolei sau prin folosirea senzorului de amprentă (dacă telefonul dispune de un astfel de senzor). Aceștia mai au posibilitatea de a urmări imaginile preluate de cameră. Un server web rulează pe sistemul încorporat administrează o bază de date cu utilizatorii înregistrați, precum și un desfășurător în care se înregistrează toate acțiunile efectuate de către utilizatori.

Lucrări asemănătoare

În [2] o placă Raspberry Pi3 cu conexiune WiFi a fost utilizată în cadrul unei aplicații IoT de securitate și de supraveghere activă a unei clădiri. Obiectivul principal este detecția focului și a intrușilor, fiind posibilă și urmărirea imaginilor preluate de cameră în timp real.

Sistemul propus în [3] reprezintă o automatizare mai extinsă pentru o casă inteligentă. Acesta folosește de asemenea o placă Raspberry Pi la care se conectează direct un senzor biometric de amprentă, împreună cu senzori de vibrație, foc, temperatură și gaz. Dacă testarea amprentei are succes, utilizatorul trebuie să se autentifice la server-ul web și să răspundă la o întrebare de securitate.

Până nu de mult, implementarea detecției și recunoașterii faciale complexe era foarte dificil de realizat pe sisteme încorporate datorită numărului mare de calcule necesare. Chiar și în prezent anumite limitări sunt impuse de puterea redusă de calcul a acestor sisteme. În [4], se prezintă un studiu care investighează fezabilitatea implementării sistemelor de recunoaștere facială în hardware special folosind sisteme FPGA (Field Programmable Gate Array).

Alte lucrări [5-10] propun câteva implementări de sisteme de control al accesului care sunt conectate la Internet direct sau folosind rețele WPAN (*Wireless Personal Area Networks*) precum ZigBee [5], *Bluetooth Low Energy* (BLE) [6] sau BodyCom [7].

Sistemul de securitate și control al accesului, propus de lucrarea de față, folosește o aplicație de tip client-server și un dispozitiv inteligent pe care majoritatea persoanelor îl dețin și îl folosesc peste tot – smartphone-ul (telefonul inteligent). Se pleacă de la premisa conform căreia există o rețea WiFi disponibilă și că aceasta este la fel de sigură ca și smartphone-ul folosit pentru a cere accesul în clădire.

Structura lucrării

Lucrarea este împărțită în patru capitole:

- **Capitolul 1** reprezintă introducerea;
- În **Capitolul 2** sunt prezentate componentele și modulele principale ale sistemului, atât hardware cât și software;
- **Capitolul 3** prezintă detaliat procesul de implementare al aplicației, cât și dificultățile întâmpinate în elaborarea acesteia;
- **Capitolul 4** este dedicat prezentării modului de funcționare al sistemului, în care se evidențiază și eventualele limitări ale aplicației; concluzionează cele prezentate în capitolele anterioare și evidențiază anumite aspecte care pot fi îmbunătățite în versiuni ulterioare ale aplicației.

2. Resurse hardware și software

Sistemul este format dintr-o placă Raspberry Pi3 model B, o camera de luat vederi, o încuietoare electromagnetică și un circuit de alimentare a acesteia. În elaborarea aplicației software, s-au folosit funcționalități puse la dispoziție de bibliotecile OpenCV și LibXML.

2.1. Placa mini PC Raspberry Pi

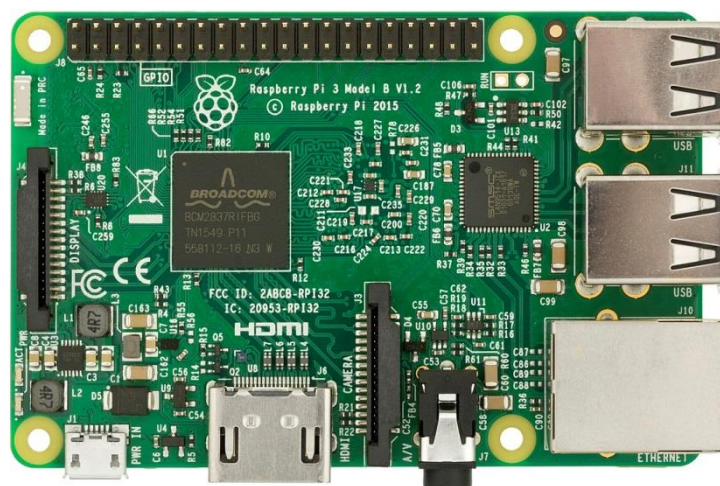


Fig. 2.1.1 – Raspberry Pi3 model B

Raspberry Pi este o serie de mini computere pe o singura placă dezvoltată de Fundația Raspberry Pi în Regatul Unit. Ea a fost concepută pentru a fi folosită în școlile din țările în curs de dezvoltare. Modelul original a devenit extrem de popular, fiind folosit în domenii precum robotică sau automatică. Nu include periferice (precum tastatură, mouse sau carcasă), deși unele accesorii au fost incluse în pachete oficiale sau neoficiale. [11]

Resurse hardware:

- Procesor Broadcom BCM2837 Quad-Core (patru nuclee), 1.2 GHz, arhitectură 64-bit și 512 kb memorie cache;
- Placă grafică integrată (GPU – *Graphics Processing Unit*) VideoCore care rulează la 400 MHz
- 1 GB memorie RAM;
- Conectare la Internet atât prin Wi-Fi 2.4 GHz cât și Ethernet 10/100 mbits/sec;
- Controler video capabil să redea până la rezoluție FullHD (1920 x 1080 pixeli) prin portul HDMI (*High Definition Multimedia Interface*);
- 4 port-uri USB 2.0;
- Jack audio 3.5mm;

- 40 pini I/O cu multiple funcții;
- 2 porturi speciale prin intermediul cărora se pot conecta periferice precum camere sau ecrane tactile (*touchscreen display*);
- Slot pentru card SD (*Secure Digital*), pentru instalarea sistemului de operare și pentru stocarea de date.

Raspberry Pi3 rulează sistemul de operare Raspbian, bazat pe distribuția de Linux, Debian Wheezy. Raspbian include peste 35,000 de pachete software pre-compilate care cuprind atât driver-e cât și alte aplicații, făcându-l extrem de bine optimizat pentru această arhitectură. Combinația dintre software-ul open-source și puterea sporită de calcul, interfațarea facilă cu diferite procese sau cu omul (HMI – *Human-Machine Interface*) la un cost redus și dimensiuni mici, fac din Raspberry Pi un candidat foarte bun pentru proiectele IoT. [11]

2.2. Camera de luat vederi

Camera de luat vederi folosită în acest proiect este Microsoft LifeCam HD-3000 (Fig. 2.2.1), conectată la placă prin USB 2.0. Aceasta are rezoluția maximă de 1280 x 720 px, iar rezoluția senzorului este de 1.3 MP. Imaginile preluate de către aplicație au rezoluția de 640 x 480 px, pentru a reduce puterea de calcul necesară în procesarea lor.



Fig. 2.2.1 – Camera Web Microsoft LifeCam HD-3000

2.3. Încuietoare electromagnetă și circuit de alimentare

Pentru încuierea și deschiderea ușii s-a instalat o încuietoare cu bobină (Fig. 2.3.1). Aceasta este alimentată la 12V curent continuu, și consumă 1,2 A. Acest tip de încuietoare se va deschide atunci când primește un impuls electric. În cazul unor pene de current, yala rămâne închisă, asigurând securitatea zonelor unde este montată.



Fig. 2.3.1 – Încuietoare electromagnetă

Datorită faptului ca pinii de uz general puși la dispoziție de placa Raspberry Pi furnizează la ieșire o tensiune de 3.3V și un curent maxim de 50mA, pentru alimentarea încuietorii s-a conceput un circuit de alimentare. Schema circuitului este prezentată în Fig. 2.3.2.

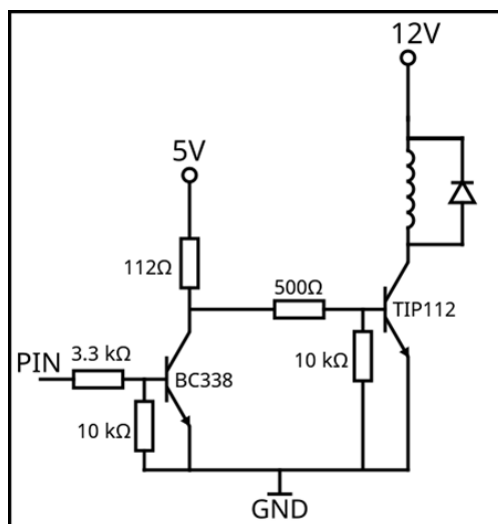


Fig. 2.3.2 – Circuit alimentare bobina electromagnetă

Circuitul din Fig. 2.3.2 are la bază doi tranzistori bipolari NPN, dintre care unul de putere (TIP112). În ansamblu, acest circuit funcționează în regim de switch cu logică inversată. Astfel, atunci când pinul microcontrolerului este în starea *LOW*, tranzistorul BC338 este blocat, iar tensiunea de 5V este aplicată în baza tranzistorului de putere. Acesta se deschide, iar bobina este alimentată și ușa se descuie. Pe de altă parte, dacă pinul microcontrolerului este în starea *HIGH*, se deschide tranzistorul BC338, iar curentul se scurge de la sursă la masă, blocând tranzistorul TIP112. Bobina nu este alimentată, și ușa este încuiată.

Dioda de decuplare, montată în paralel cu bobina, are rolul de a prelua curentul acumulat în bobină la decuplarea tensiunii, protejând astfel tranzistorul.

Fig 2.3.3 reprezintă circuitul realizat fizic:

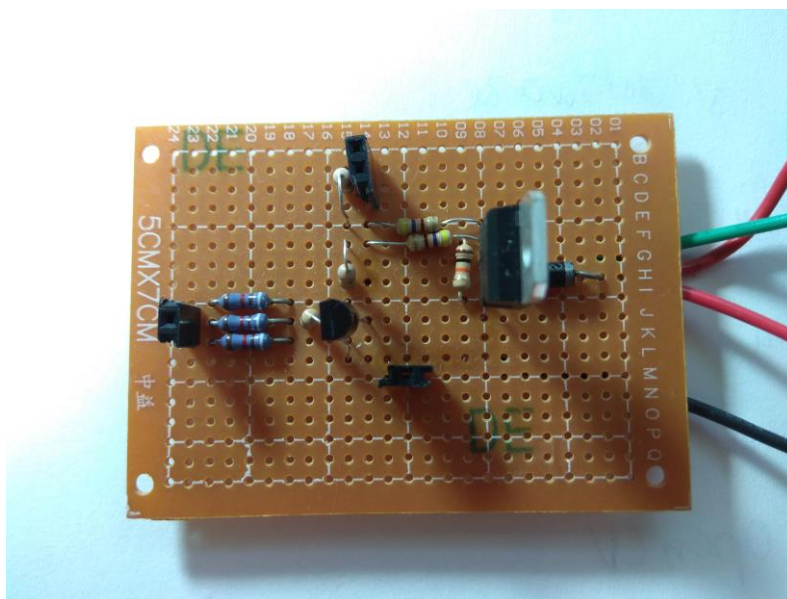


Fig. 2.3.3 – Circuitul de alimentare a încuietorii electromagnetice

2.4. Biblioteca OpenCV

În secțiunile 2.1, 2.2 și 2.3 au fost prezentate resursele hardware care compun sistemul încorporat de securitate și de control al accesului. Această secțiune prezintă biblioteca utilizată în elaborarea acestui proiect pentru preluarea, prelucrarea, afișarea și codarea/decodarea imaginilor preluate de la camera video.

OpenCV (*Open Source Computer Vision*) este o bibliotecă de funcții create în special pentru aplicații de procesare de imagini în timp real. Librăria este disponibilă pe mai multe platforme printre care Windows, MacOS, Linux Android, iOS sau Blackberry, și ea este disponibilă gratuit sub licența open-source BSD.

Lansat pentru prima dată în anul 1999, proiectul OpenCV a fost inițial o inițiativă a firmei Intel pentru a avansa aplicațiile cu cerințe mari de procesare. Prima versiune a fost făcută disponibilă publicului larg în anul 2000, în cadrul *IEEE Conference on Computer Vision and Pattern Recognition*. În prezent, ultima versiune lansată este 3.4.0.

Librăria a fost dezvoltată în limbajul C++, dar au fost dezvoltate port-uri pentru alte limbaje de programare precum C#, Java, Python, MATLAB, Perl, Haskell sau Ruby.

Principalele arii de aplicabilitate pentru OpenCV sunt:

- Procesarea 2D și 3D;
- Recunoaștere facială;
- Recunoașterea gesturilor;
- Robotică;
- Identificarea obiectelor;
- Realitate augmentată;
- Stereoscopie;
- Localizarea obiectelor în mișcare.

Pentru a susține unele din ariile de mai sus, OpenCV include și o librărie de *machine learning* care conține Clasificator Bayes, Rețele neuronale artificiale etc.

În cadrul acestui proiect au fost utilizate funcționalitățile de accesare a camerei de luat vederi, de prelucrare a imaginilor preluate (îmbunătățire a contrastului și a luminozității), de detecție facială și de afișare a imaginilor rezultate în urma acestor prelucrări.

2.5. Biblioteca LibXML

Limbajul XML (*Extensible Markup Language*) este un meta-limbaj de marcare recomandat de Consorțiul Web (*World Wide Web Consortium*) pentru crearea de alte limbaje de marcare precum XHTML, SVG etc. XML este o simplificare a limbajului SGML (*Standard Generalized Markup Language*), din care se trage și HTML.

Folosit pentru prima dată în anul 1996, XML definește un set de reguli pentru codarea documentelor într-un format care poate fi ușor de citit atât de oameni cât și de mașină și este caracterizat prin simplitate și generalitate. Un document XML este format, în general, din trei componente:

- **Eticheta (Tag)** – este un constructor de marcare care începe cu caracterul „<” și se termină cu caracterul „>”. Etichetele pot fi de trei tipuri:
 - Eticheta de start, precum `<secțiune>`;
 - Eticheta de stop, precum `</secțiune>`;
 - Etichetă nulă (nu intră în componența unui element). Acest tip de tag îndeplinește funcții specifice, de exemplu sfârșit de linie.
- **Element** – Elementul este o componentă logică a documentului care, de regulă începe cu o etichetă de start, și se încheie cu o etichetă de stop, sau este format dintr-o etichetă nulă. Caracterele aflate între etichetele de start și de stop constituie conținutul elementului și poate fi un șir de caractere sau alte elemente, numite *elemente-copil*. Exemplu: `<secțiune> Numele secțiunii </secțiune>`;
- **Atribut** – Un atribut reprezintă o pereche *nume-valoare* care îi este atribuită unui element, și poate exista doar în cadrul unei etichete start. Exemplu: `<secțiune numar="3"> Numele secțiunii </secțiune>` [14]

Cu acest set de reguli, informațiile pot fi structurate foarte ușor, iar acest lucru s-a dovedit a fi foarte util în schimbul de informații prin Internet. Deoarece XML este folosit pe scară largă în prezent, a apărut necesitatea unor biblioteci de funcții care să faciliteze codarea și extragerea informațiilor din XML pentru a le putea procesa. Una din bibliotecile foarte mult utilizate în mediul practic este LibXML.

LibXML este o bibliotecă software open-source specializată în analiza fișierelor XML. Ea permite crearea, citirea, scrierea fișierelor XML precum și adăugarea și ștergerea de elemente din cadrul acestora cu consum mic de resurse și cu viteză mare. LibXML este scris în limbajul C, de către dezvoltatorii proiectului *GNOME*. Prima versiune a apărut în anul 1999 și a fost într-o continuă dezvoltare până în prezent. Deși este scris în limbajul C, există port-uri către toate limbajele de programare uzuale (C++, Java, C#, Pascal, Ruby, Python, PHP, etc.) [15] [16]

În cadrul proiectului au fost folosite funcționalitățile de scriere și citire a fișierelor XML.

3. Implementarea proiectului

Așa cum s-a precizat în primul capitol, această lucrare propune un sistem încorporat de securitate și control al accesului în clădire. Acest sistem este construit în întregime în jurul plăcii Raspberry Pi, și include o camera video precum și o încuietoare electromagnetică. Utilizatorii sistemului vor interacționa cu acesta prin intermediul unei aplicații mobile, care va fi prezentată într-un capitol viitor. Fig 3.0.1 reprezintă arhitectura generală a sistemului.

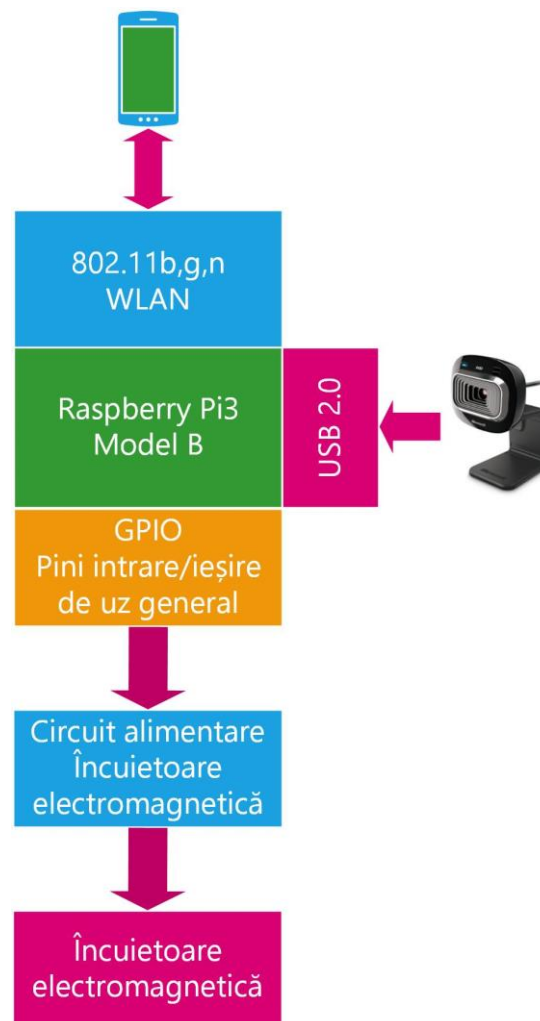


Fig. 3.0.1 – Arhitectura generală a sistemului de securitate

După cum se poate observa în Fig. 3.0.1, utilizatorul comunică cu sistemul prin intermediul aplicației mobile instalate pe un dispozitiv Android. Comunicația dintre telefon și placa Raspberry Pi se face prin Internet Wireless. Camera video captează imagini pe care le trimite la placă prin USB2.0. Imaginile pot fi transmise înapoi la utilizator, sau folosite intern în luarea anumitor decizii. Atunci când utilizatorul dorește, se trimite o comandă de descuieră a ușii. Sistemul solicită autentificarea utilizatorului (prin reintroducerea parolei sau prin folosirea senzorului de amprentă), și descuie ușa pentru o perioadă de timp prestabilită de administrator.

Se pot distinge din Fig. 3.0.1 două aplicații distincte: aplicația care rulează pe smartphone și cea care rulează pe placa Raspberry Pi3. Deoarece arhitectura sistemului este proiectată după modelul *client-server*, în această lucrare aplicația mobilă va fi numită pe scurt „client”, iar aplicația care rulează pe placă va fi numită „server”.

3.1. Aplicația server

Aplicația care rulează pe placa Raspberry Pi3 (aplicația server) are următoarele cerințe de îndeplinit:

- Preluarea imaginilor de la cameră;
- Procesarea imaginilor și detecția fețelor;
- Afișarea imaginilor;
- Administrarea bazei de date cu utilizatori;
- Administrarea bazei de date cu acțiunile desfășurate de aceștia;
- Comunicația cu aplicația mobilă (cu posibilitatea comunicației cu mai mulți utilizatori simultan);
- Transmiterea de imagini către utilizatorii care doresc acest lucru;
- Încuierea și descuiera uşii atunci când utilizatorul o cere.

Aplicația a fost dezvoltată în limbajul C++ pe un PC (procesor Intel Core i3, 8 GB RAM) cu sistemul de operare Ubuntu 16.04, nucleul Linux 4.13, în mediul de dezvoltare Eclipse Oxygen 4.7.2, iar compilarea s-a efectuat cu compilatorul GCC 5.4. S-a ales dezvoltarea pe PC din cauza dificultăților întâmpinate la rularea sesiunilor de depanare pe placa Raspberry Pi, dificultăți generate de resursele limitate ale plăcii (atât în ceea ce privește memoria cât și microprocesorul).

3.1.1. Împărțirea aplicației pe fire de execuție

Date fiind cerințele aplicației, enumerate în secțiunea anterioară, o funcționare de timp real a fost absolut necesară pentru a îndeplini toate obiectivele într-un timp rezonabil. Faptul că Raspberry Pi3 nu pune la dispoziția utilizatorului un ceas de timp real, a făcut imposibilă utilizarea unui sistem de operare de timp real convențional precum FreeRTOS, Zephyr sau NuttX, deoarece aceste sisteme depind de existența unui astfel de circuit. Ceea ce oferă însă Raspberry Pi este posibilitatea rulării unui sistem de operare Linux, lucru care constituie un mare avantaj al acestei plăci față de soluțiile oferite de alți producători, deoarece nucleul Linux dă posibilitatea dezvoltării în mai multe limbaje de programare. Funcționarea de timp real a fost emulată prin folosirea clasei *pthread* (fir de execuție) din biblioteca standard C.

Una din cele mai puternice caracteristici ale sistemului de operare Linux este suportul multitasking, adică rularea simultană în memorie a mai multor procese. Un proces este obiectul care deține toate resursele unei aplicații. Un proces Linux poate crea unul sau mai multe fire de execuție. Un fir de execuție este o cale independentă de execuție în cadrul unui proces cu care firul partajează spațiul de adresă, codul și datele globale. Fiecare fir posedă propria stivă, propriile mecanisme de intrare, inclusiv o coadă privată de mesaje. De fiecare dată când Linux inițializează o nouă instanță a unui proces, sistemul de operare creează un nou fir primar pentru acel proces. Firul primar pornește odată cu încărcarea programului în Linux. Pentru cele mai multe aplicații, firul primar creat de sistemul de operare este singurul fir cerut de aplicație. Cu toate acestea, pentru a-și ușura operarea, procesele pot crea fire

suplimentare. Ideea care stă la baza creării de fire suplimentare este folosirea la maxim și cât mai eficient, a timpului de prelucrare al unității centrale. [15]

Există o serie de reguli de care trebuie să se țină seama atunci când se optează sau nu pentru crearea firelor multiple:

- Cu foarte rare excepții, toate componentele interfeței utilizator trebuie să partajeze un fir comun;
- Programele trebuie să creeze fire în măsura în care este nevoie de ele și să evite crearea de fire suplimentare și „ținerea lor în rezervă”;
- Programele trebuie să elibereze firele după terminarea procesului lor de prelucrare;
- Aplicațiile mai complexe care utilizează ferestre multiple, pot solicita fire suplimentare pentru administrarea prelucrărilor în cadrul anumitor ferestre;
- Nu trebuie create fire care permit utilizatorului să întrerupă un proces important al sistemului, care poate avea drept consecințe coruperea memoriei sau a altor prelucrări curente.

Linux acceptă executarea de fire multiple (*multithreading*). Într-un mediu în care unul sau mai multe fire se execută concurențial, va fi important ca programele să poată sincroniza activitățile diferitelor fire. Sistemul de operare furnizează o serie de facilități de sincronizare care permit firelor să-și sincronizeze acțiunile cu alte fire.

În general, un fir se sincronizează cu un altul prin „adormire” (*sleep*). Când un fir este adormit, sistemul de operare nu-i mai programează timp CPU (*Central Processing Unit*), iar firul se oprește din execuție. Înainte însă de a adormi, firul comunică sistemului de operare ce „eveniment special” trebuie să apară pentru a intra din nou în execuție (o acționare de tastă, clic de mouse sau încheierea unui algoritm). [15]

Linux acceptă mai multe tipuri diferite de obiecte de sincronizare. Dintre acestea, cele mai utilizate sunt: secțiunea critică, excluderea mutuală, semaforul, evenimentul și timp de așteptare.

- Secțiunea critică (*critical section*) – Reprezintă o secvență mică de cod care cere acces exclusiv la o serie de date partajate, înainte ca respectivul cod să poată începe execuția;
- Excluderea mutuală (*Mutex*) – Se aseamănă mult cu o secțiune critică, însă ea este folosită pentru sincronizarea accesului la date în procesele multiple;
- Semaforul (*Semaphore*) – Programele vor folosi obiecte semafor pentru contorizarea resurselor. Numai un singur fir poate folosi un semafor la un moment dat;
- Evenimentul (*Event*) – Obiectele eveniment sunt cele mai primitive obiecte de sincronizare și sunt foarte diferite de excluderile mutuale și de semafoare. Programele vor utiliza evenimente pentru semnalarea îndeplinirii unei operații. De regulă, evenimentele sunt utilizate pentru a porni un alt fir după ce firul curent efectuează o parte din prelucrarea sa;
- Timp de așteptare (*Waiting time*) – Sunt utilizate pentru a comunica sistemului de operare (înainte ca firul să intre în „adormire”) după cât timp un fir este gata de a intra din nou în execuție. [15]

Folosind fire de execuție, aplicația a fost împărțită în șase subtask-uri, astfel:

- Firul **main** inițializează aplicația, crează celelalte fire de execuție și preia imagini de la cameră pe care le adaugă într-o coadă de mesaje;
- Firul **detect** preia imagini din coada în care adaugă main, și caută fețe. Dacă există fețe, le încadrează într-un dreptunghi, iar imaginea rezultată o adaugă în altă coadă. Dacă nu găsește nici o față, intră în starea de adormire până la apariția unei noi imagini;
- Firul **print** va prelua imagini din cele doua cozi menționate anterior si le va afișa pe ecran (cu prioritate pe cele în care fețele sunt detectate). Acest fir este util doar în faza de dezvoltare a aplicației. În versiunea finală va fi exclus deoarece el este util doar în prezența unui monitor;
- Firul **serverTCP** este cel responsabil de comunicația cu aplicația mobilă. Va primi comenzi și va raspunde în consecință;
- Firul **serverUDP** va trimite imagini (neprocesate) către utilizatorii care solicită acest lucru, simultan;
- Firul **door** va descuia / încuia ușa atunci cand primește comanda de la firul serverTCP.

Fig. 3.1.1.1 prezintă cele șase fire și execuția acestora în paralel.

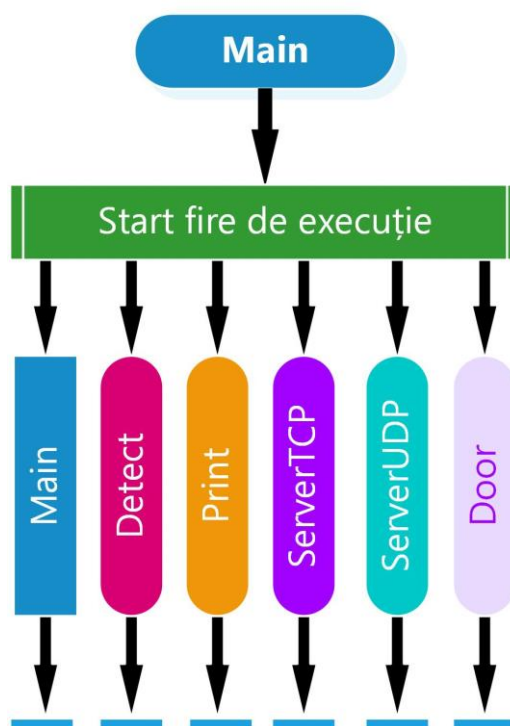


Fig. 3.1.1.1 – Arhitectura pe fire multiple ale aplicației server

Figurile Fig. 3.1.1.2 și Fig. 3.1.1.3 reprezintă schemele bloc ale aplicației server, unde se poate observa felul în care comunică cele șase fire între ele.

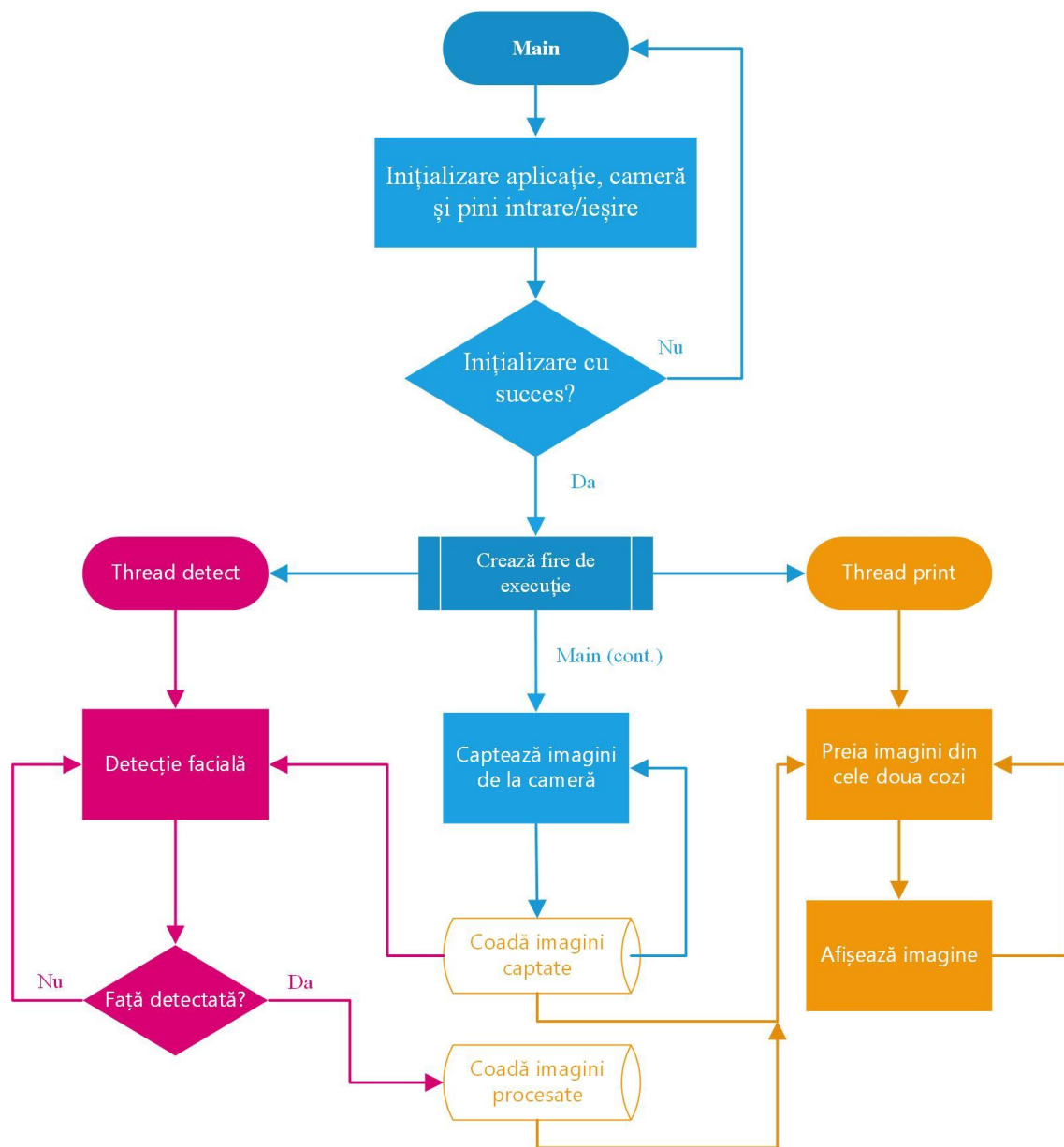


Fig. 3.1.1.2 – Implementarea firelor de execuție *main*, *detect* și *print*

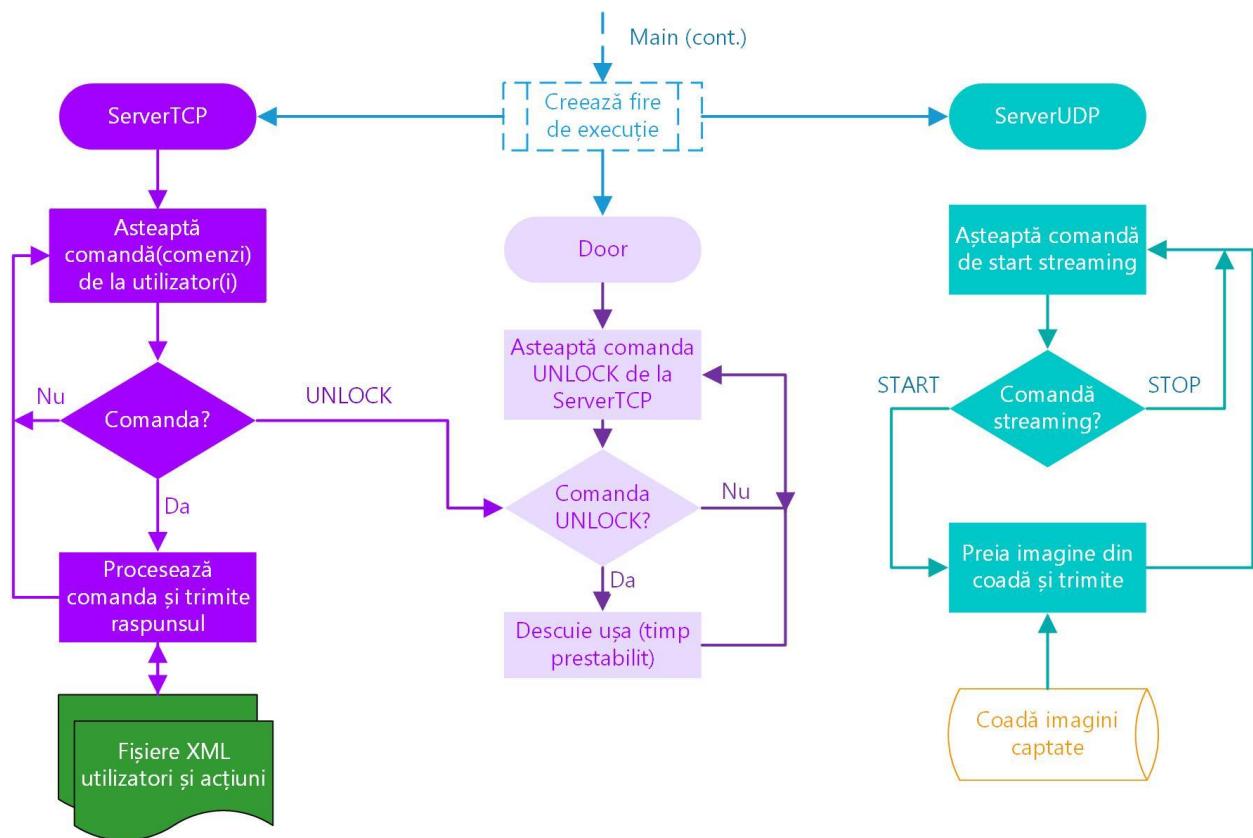


Fig. 3.1.1.3 – Implementarea firelor de execuție *serverTCP*, *door* și *serverUDP*

Așa cum se poate observa în figurile Fig. 3.1.1.2 și Fig. 3.1.1.3, firele se execută independent unul față de celălalt, iar sincronizarea lor se face fie prin cozi de mesaje, fie prin evenimente de tipul comenzi primite de la utilizator sau de la alt fir de execuție.

3.1.2. Preluarea și procesarea imaginilor (main, detect și print)

Procesarea digitală a imaginilor reprezintă utilizarea de algoritmi pentru a aduce modificări sau pentru a extrage anumite informații din imagini digitale. Ca o subcategorie a procesării de semnale digitale, procesarea imaginilor digitale are multe avantaje față de procesarea imaginilor analogice.

Multe din tehnicile de prelucrare de imagini digitale au fost dezvoltate în anii '60 la *Jet Propulsion Laboratory*, la *Massachusetts Institute of Technology (MIT)*, la *Bell Laboratories*, *University of Maryland* sau alte institute de cercetare cu aplicații în astronomie, medicină, sau îmbunătățirea fotografiilor. Costul de procesare era destul de ridicat la acea vreme, datorită puterii reduse de calcul. Acest lucru avea să se schimbe în anii '70, atunci când au fost lansate pe piață mașini de calcul mai ieftine și hardware dedicat pentru procesarea de imagini. Imaginile au putut fi procesate în timp real, ceea ce a deschis orizonturi spre multe alte domenii.

În prezent, procesarea de imagini este utilizată extensiv în explorările spațiale (de exemplu roboții trimiși de NASA pe Marte folosesc camere de luat vederi pentru a explora planeta roșie), în medicină (toate tehnologiile de imagistică medicală depind de procesarea de imagini pentru a ușura munca personalului medical în stabilirea diagnosticului), în biometrie (detectia și recunoașterea facială, recunoașterea irisului sau recunoașterea amprentelor), în domeniul inteligenței artificiale (mașinile autonome depind de sisteme de vedere artificială pentru a recunoaște obstacole, oameni sau alte vehicule apropiate).

În funcție de aplicație, procesarea de imagini are mai multe etape:

- Achiziția imaginii
- Preprocesarea imaginii
- Segmentarea imaginii
- Descrierea
- Recunoașterea formelor
- Interpretarea

Etapă de achiziție constă în utilizarea unei camere de luat vederi pentru a prelua o imagine și de a o stoca în memorie. Preprocesarea este etapa în care se filtrează zgomotul și se identifică muchiile și colțurile. Prin segmentare se înțelege separarea obiectelor din imagine de fundal. Descrierea și recunoașterea formelor sunt etapele responsabile de conturul obiectelor și de calcularea anumitor proprietăți ale acestora cum ar fi aria, perimetrul, compactitatea, centrul de greutate ș.a. Interpretarea este ultima etapă a procesării de imagini. Ea are ca scop identificarea diferitelor forme geometrice cum ar fi dreptunghiuri sau cercuri, litere și cifre din imagine sau identificări mai complexe precum recunoașterea facială sau identificarea obstacolelor. [12] [13]

Detectia facială este un algoritm care face parte din domeniul procesării de imagini, și constă în identificarea fețelor umane în imaginile digitale. Tehnologia biometrică utilizează caracteristici ale corpului uman cum ar fi caracteristici ale feței, ampretei, vocea sau irisul. În detectia fețelor, nu este necesar contactul fizic cu senzorul (cum este cazul amprentelor), iar imaginea poate fi capturată foarte facil utilizând o camera de luat vederi. Acest lucru face ca detectia feței să fie una dintre cele mai robuste metode de identificare a persoanelor, deoarece nu are nevoie de condiții speciale. Este importantă etapa de detecție, deoarece ea

reprezintă primul pas în diferite aplicații precum recunoașterea fețelor, supraveghere video, interacțiune dintre mașina și om, etc.

În trecut, echipamente dedicate și foarte scumpe erau utilizate pentru detecția în timp real a fețelor. Chiar și pe echipamentele existente în prezent, detecția durează foarte mult, în special atunci când un număr mare de imagini este folosit. De aceea procesul de detecție a fețelor trebuie accelerat. Primul algoritm capabil să detecteze fețe umane în timp real este algoritmul Viola-Jones. Acest algoritm a devenit procedura standard folosită în astfel de aplicații, deoarece are un timp de calcul redus, însă nu este indicat în cazul imaginilor cu rezoluții mari. Problema detecției faciale este foarte simplă pentru o ființă umană, însă o mașina de calcul are nevoie de instrucțiuni precise și constrângeri. Pentru a reduce complexitatea problemei, algoritmul are ca cerință o fotografie cu fața frontală și verticală. La prima vedere s-ar părea că aceste constrângeri diminuează utilitatea aplicației, dar, pentru că etapa de detecție este de multe ori urmată de o etapă de recunoaștere, aceste limitări sunt acceptabile.

Așa cum s-a menționat în capitolul anterior, firul de execuție **main** inițializează aplicația și tot el preia imagini de la camera video, pe care le depune într-o coadă. Dacă inițializarea eșuează, aplicația se oprește. Dacă toate componentele sistemului funcționează corect, **main** continuă cu crearea celorlalte fire de execuție, pornind astfel toate modulele aplicației.

```
if(initApplication() == 0) //Dacă initializarea are succes...
{
    //Se creează firele de execuție
    errThread = pthread_create(&ThreadDetect, NULL, &detectAndDisplay, NULL);
    if (errThread != 0) //Dacă firul a apărut o eroare la crearea firului
    {
        exit(1);
    }
    errThread = pthread_create(&ThreadPrint, NULL, &print, NULL);
    if (errThread != 0)
    {
        exit(1);
    }
    errThread = pthread_create(&ThreadServer, NULL, &runTcpServer, NULL);
    if (errThread != 0)
    {
        exit(1);
    }
    errThread = pthread_create(&ThreadUDP, NULL, &runUdpServer, NULL);
    if (errThread != 0)
    {
        exit(1);
    }
    errThread = pthread_create(&ThreadDoor, NULL, &runDoor, NULL);
    if (errThread != 0)
    {
        exit(1);
    }
}
```

Fig. 3.1.2.1 – Rutină pentru crearea firelor de execuție

Aşa cum se poate observa în secvența de cod prezentată în Fig. 3.1.2.1, dacă apare o problemă la inițializarea aplicației sau la crearea oricarui fir de execuție, aplicația se oprește.

```
VideoCapture cap(0);
if(!cap.isOpened())
{
    cout<<"Could not open the webcam"<<endl; /* eroare la deschidere camera */
    exit(1); /* Inchide aplicatia cu eroare */
}
else{
    cout<<"Camera is opened"<<endl; /* Succes deschidere camera -> Aplicatia poate porni
    */
}
while(1)
{
    if(cap.read(frame)) /* se citeste frame-ul */
    {
        if(qFetch.size() == limQFetch) //Daca s-a umplut coada, se goleste
        {
            qFetch.erase(qFetch.begin(),qFetch.begin()+qFetch.size()-1);
        }
        qFetch.push_back(frame); /* Se adauga in coada de frame-uri neprocesate */
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(30)); /* thread-ul main
    cedeaza procesorul */
}
```

Fig. 3.1.2.2 – Bucla infinită în care se captează imagini de la cameră și sunt depuse în coadă

Figura 3.1.2.2 reprezintă funcția pe care o execută firul de execuție main după ce inițializează aplicația. Main accesează camera prin intermediul unui obiect de tipul **cv::VideoCapture** (clasă din biblioteca OpenCV) și citește câte o imagine la fiecare 30 de milisecunde, pe care o depune în coada **qFetch**. Thread-ul **detect** va prelua imagini din această coadă și va căuta fețe. Dacă va găsi, le va încadra într-un chenar de culoare albastră și le va depune în alta coadă (**qDetect**), așa cum se poate vedea în Fig. 3.1.2.3.

```

if(!qFetch.empty())
{

    frame = qFetch.front(); /*In aceasta variabila va fi salvat ultimul frame adaugat in
                                coada qFetch */
    qFetch.pop_back(); /* Ultimul frame adaugat se sterge din coada */
    face_cascade.detectMultiScale(frame, faces, 1.1, 3); // detectia fetelor din imagine.

    if(faces.size() > 0) // Daca s-au detectat fete...
    {
        for(unsigned int ic = 0 ; ic < faces.size() ; ic ++)
        {
            Point *pt1 = new Point(faces[ic].x,faces[ic].y);
            Point *pt2 = new Point( (faces[ic].x + faces[ic].width) , (faces[ic].y
+                               faces[ic].height) );
            rectangle(frame,*pt1,*pt2,Scalar(255, 0, 0), 5, 8, 0);

            delete pt1;
            delete pt2;
        }

        if(qDetect.size() < limQDetect) /* Daca coada qDetect nu este plina */
        {
            qDetect.push_back(frame);
            /* Se adauga imaginea in coada si urmeaza sa fie afisata de thread-ul
de            Print */
            faces.empty(); /* Se goleste vectorul fetelor pentru urmatorul frame */
        }else
        {
            qDetect.erase(qDetect.begin(),qDetect.begin()+qDetect.size()-1);
        }
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(30));
}

```

Fig. 3.1.2.3 – Funcția de detecție a fețelor și adăugarea imaginilor care conțin fețe în coada *qDetect*

În Figura 3.1.2.3 se poate observa faptul că detecția fețelor se face folosind funcția **cv::detectMultiScale** din biblioteca OpenCV. Această funcție are la bază algoritmul Viola-Jones cu identificatori Haar. Pentru ca detecția să fie fiabilă, este necesar ca fața să fie frontală și verticală. Algoritmul Viola-Jones are ca scop discriminarea ferestrelor, ce conțin numai o față, de celelalte ferestre dintr-o imagine. Acest lucru este efectuat prin utilizarea unor trăsături de tip Haar și calculul cât mai rapid al acestora. Fig. 3.1.2.4 arată rezultatul returnat de acest algoritm. Se poate observa faptul că fețele sunt identificate, iar celelalte regiuni din imagine sunt ignorate.

Identificatorii Haar sunt de fapt măști, trăsături simple care, prin valoarea lor, indică prezența unei fețe, pe baza unor proprietăți comune ale tuturor fețelor precum:

- Regiunea ochilor este mai întunecată decât partea superioară a obrazilor;
- Șaua nasului este mai luminoasă decât zona ochilor; [13]

Fig. 3.1.2.5 arată identificatorii Haar cel mai des folosiți în aplicațiile de detecție a fețelor.

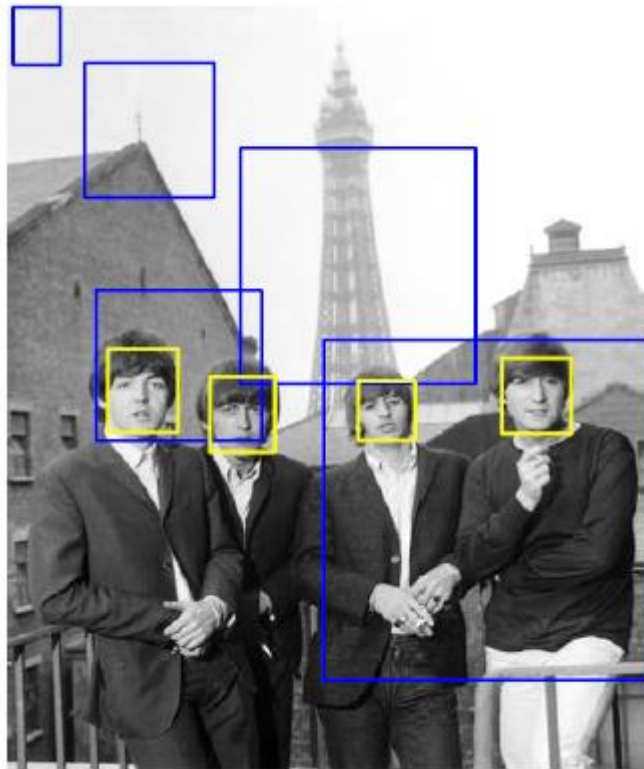


Fig. 3.1.2.4 – Rezultatul algoritmului Viola – Jones [13]

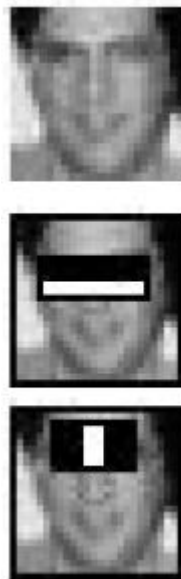


Fig. 3.1.2.5 – Exemple de identificatori Haar folosiți în aplicațiile de detecție a fețelor

Firul de execuție **print** verifică dacă există imagini în coada *qDetect*, și le afișează. Dacă nu există, afișează pe ecran imaginile preluate de firul **main**, așa cum se poate vedea în Fig. 3.1.2.6.

```
while(1)
{
    if(!qDetect.empty() && !faces.empty()) /* Daca exista frame-uri in coada
qDetect... */
    {
        frame = qDetect.front();
        qDetect.pop_back();
        imshow("Display window",frame); /* ...se afiseaza ultimul adaugat */
        waitKey(1);

    }
    else if(!qFetch.empty() && faces.empty())
    {
        /* TODO de vazut o modalitate de a afisa imagine de la camera fara
ca o persoana sa
        * fie in cadru sau nu */
        frame = qFetch.front();
        qFetch.pop_back();

        imshow("Display window",frame); /* ...se afiseaza ultimul adaugat */
        waitKey(1);
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(30));
}
```

Fig. 3.1.2.6 – Funcția de afișare a imaginilor (au prioritate cele în care sunt detectate fețe)

3.1.3. Firul de execuție ServerTCP

Protocolul TCP (*Transmission Control Protocol*) este un protocol de comunicație orientat pe conexiune nod la nod, în care pentru fiecare pachet transmis se așteaptă o confirmare din partea echipamentului de destinație. Transmisia următorului pachet nu se realizează până când nu se primește confirmarea pentru pachetul transmis anterior. Se asigură atât trimiterea cu succes a tuturor pachetelor cât și recepția acestora în ordinea în care au fost trimise. O conexiune este o legătură logică ce apare între două entități care comunică prin intermediul unei rețele de calculatoare (sau Internet). Conexiunea apare atunci când un dispozitiv (numit *client*) solicită, prin intermediul rețelei, un serviciu de la un alt dispozitiv (numit *server*). Dacă această conexiune nu se stabilește în prealabil, comunicația nu poate avea loc, în cazul acestui protocol. Conexiunea se încheie atunci când unul din cele două dispozitive (de regula clientul) decide că aceasta nu mai este utilă. Fig. 3.1.3.1 prezintă cum are loc interacțiunea dintre cele două dispozitive (entități) care participă în sesiunea de comunicație. [16] [17]

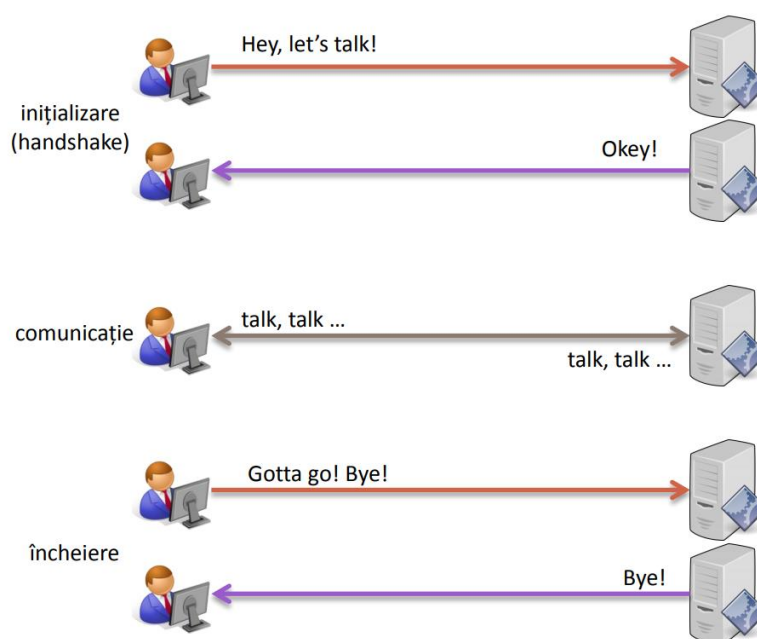


Fig. 3.1.3.1 – Sesiune de comunicație în protocolul TCP (*Transmission Control Protocol*) [16]

Așa cum s-a precizat în Capitolul 3.1.1, firul de execuție **serverTCP** este cel responsabil de comunicația cu clienții (utilizatorii înregistrați în sistem). Acest server trebuie să asigure facilitarea comunicării mai multor clienți simultan, și să răspundă la toate cererile pe care le primește. În momentul în care un client solicită conexiunea la server, acest fir de execuție creează un fir separat de execuție care să răspundă cererilor venite de la respectivul client. Conexiunea se încheie de către client, atunci când acesta dorește. Firul suplimentar creat este distrus în momentul încheierii conexiunii (când utilizatorul închide aplicația). Astfel, este asigurată facilitarea conectării mai multor clienți simultan. Tot responsabilitatea acestui thread este aceea de a administra cele două baze de date menționate în Capitolul 3.1.1

(fișiere XML; baza de date cu utilizatorii sistemului și desfășurătorul cu toate acțiunile desfășurate de aceștia).

În cadrul dezvoltării acestui server, s-au întâmpinat dificultăți în ceea ce privește împărțirea datelor în pachete și la alocarea corectă de memorie pentru fiecare pachet. Acest protocol, fiind orientat pe conexiune, clientul și server-ul se sincronizează automat, deoarece se așteaptă unul pe celalalt. De menționat este faptul că a fost necesară implementarea unor secțiuni de excludere mutuală pentru accesul la fișierele ce conțin informații despre utilizatori și acțiunile efectuate în sistem, deoarece, la un moment dat pot exista doi sau mai mulți clienți care să dorească să facă modificări acestor fișiere.

Se pot distinge, astfel, doua bucle care compun acest server:

- Buclea principală – Secțiunea de cod executată de firul **serverTCP**, în care se va aștepta o conexiune de la un client, și se va crea fir separat pentru administrarea cererilor venite de la acesta (Fig. 3.1.3.2);
- Buclea secundară – Secțiune de cod executată de firele create de **serverTCP**, în care se va stabili tipul solicitării și se va trimite răspuns (Fig. 3.1.3.3).

```
while(1)
{
    printf("server: waiting for connections...\n");

    sin_size = sizeof their_addrTcp;

    connfdTcp = accept(sockfdTcp, (struct sockaddr *)&their_addrTcp, &sin_size);
    //Dacă se primește cerere de conexiune la server
    if(connfdTcp < 0)
    {
        printf("Error in accept\n");
        continue;
    }

    inet_ntop(their_addrTcp.ss_family, get_in_addr((struct sockaddr
*)&their_addrTcp), sTcp, sizeof sTcp);
    printf("server: got connection from %s\n", sTcp);

    errThread = pthread_create(&tid, NULL, &manageClient, &connfdTcp);
    //Creează thread separat pentru clientul care a solicitat conexiune
    if (errThread != 0)
    {
        printf("\nCan't create thread :[%s]", strerror(errThread));
    }
}
```

Fig. 3.1.3.2 – Buclea principală a firului de execuție **serverTCP**

```

while(1)
{
    char* buffer;
    buffer = receiveData(connfdLocal);

    if(buffer == NULL)
    {
        close(connfdLocal);
        break;
    }

    string sBuffer = buffer;
    string username, password;
    User mUser;
    free(buffer);

    size_t spatiu = sBuffer.find_first_of(' ');
    string tipSolicitare = sBuffer.substr(0,spatiu);
    sBuffer = sBuffer.substr(spatiu+1);

    if( (tipSolicitare.compare(ADMINISTRATOR_GET_USERS)!=0) &&
        (tipSolicitare.compare(ADMINISTRATOR_USERS_CHANGED)!=0) )
    {

        spatiu = sBuffer.find_first_of(' ');
        username = sBuffer.substr(0,spatiu);
        password = sBuffer.substr(spatiu+1);

        mUser = getUserWith(username,password);
    }

    char *message = 0;
    if(tipSolicitare.compare(LOGIN_USER_COMMAND) == 0)
    {

        if(mUser.getUsername().compare(username)==0 &&
mUser.getPassword().compare(password)==0 && username.compare("")!=0 && password.compare("")!=0)
        {
            message = prepareMesaj((string)LOGIN_USER_RESPONSE_OK + " " +
mUser.getFirstName() + " " + mUser.getLastName() + " " + bool2string(mUser.isAdministrator()));
            logAction(LOG_USER_LOGIN,mUser);
        }else
        {
            message = prepareMesaj(LOGIN_USER_RESPONSE_FAILED);
        }

    }else if(tipSolicitare.compare(HOME_OPEN_DOOR_BUTTON) == 0)

    }

    //.....

```

Fig. 3.1.3.3 – Secvență de cod ce reprezintă stabilirea tipului de solicitare și conceperea răspunsului

3.1.4. Firul de execuție serverUDP

Spre deosebire de TCP, UDP sau *User Datagram Protocol*, este un protocol de comunicație care nu este bazat pe conexiune, și nu asigură transmiterea cu succes a pachetelor de date și nici transmiterea lor în ordine, pentru că nu există mesaje de confirmare (Fig. 3.1.4.1) ca la TCP. Pachetele sunt trimise pur și simplu în rețea la adresa specificată, dar nu există nici o garanție a faptului ca receptorul așteaptă sau va primi pachetul respectiv. Un avantaj al protocolului UDP față de TCP este acela al vitezei de transmisie. Dacă la TCP sunt necesare trei mesaje pentru stabilirea conexiunii și apoi pentru fiecare pachet este transmis un mesaj suplimentar de confirmare, la UDP doar pachetele utile încarcă rețeaua. Un alt avantaj al protocolului UDP este posibilitatea de a transmite la mai multi receptori în același timp (*multicasting*), lucru imposibil de realizat în cazul protocolului TCP.

Pe de altă parte, un dezavantaj al UDP față de TCP este acela că nu există mecanisme de detectare a erorilor, acest lucru căzând în sarcina programatorului, iar TCP se ocupă automat atât de detecția erorilor cât și de retransmiterea pachetelor compromise.

Principalele arii de folosință ale acestui protocol sunt transmiterile de date multimedia (fișiere video, fișiere audio), acolo unde pierderea unui pachet nu este crucială. [16] [17]

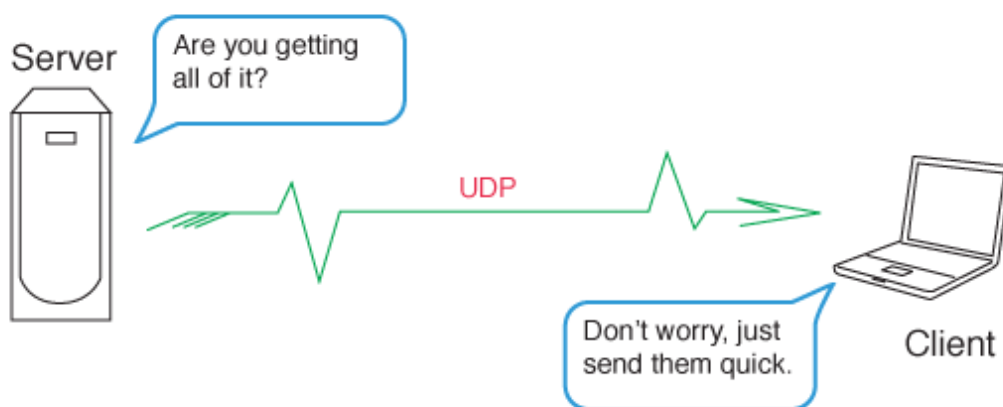


Fig. 3.1.4.1 – Transmiterea de pachete prin UDP; Lipsa mesajelor de confirmare [18]

Așa cum s-a menționat în Capitolul 3.1.1, firul de execuție **serverUDP** așteaptă o comandă de la utilizatorii care doresc să urmărească imaginile captate de camera video instalată în sistem. Inițial s-a dorit implementarea mecanismului de multicasting (acesta fiind și motivul pentru care s-a optat pentru UDP), dar, datorită multiplelor dificultăți întâmpinate la sincronizarea dintre client și server, cât și la codarea și decodarea imaginilor (din format binar în JPEG, și invers), timpul nu a mai permis.

serverUDP așteaptă o comandă de *start* de la client și transmite imaginile în ordine până când primește o comandă de *stop*.

Înainte de transmitia efectivă a pachetelor, pentru a asigura sincronizarea clientului cu serverul la nivel de imagine, se transmit (și se așteaptă confirmarea primirii) dimensiunea în octeți a imaginii și numărul total de pachete care urmează a fi trimise. Astfel, receptorul va ști când a primit toate pachetele ce compun o anumită imagine.

Înainte de a fi împărțite în pachete și trimise, imaginile sunt codate în formatul JPEG pentru a reduce dimensiunea lor. Codarea se efectuează prin intermediul funcției **cv::imencode** din biblioteca OpenCV. Au fost întâmpinate dificultăți la decodarea imaginilor, după recepția lor, probabil din cauza unor erori de transmisie a datelor prin UDP. (Fig. 3.1.4.2)

```
while (1)
{
    if(streaming && !qFramesForStreaming.empty())
    {
        begin = 0;
        end = 0;
        currentFrame = qFramesForStreaming.back();
        qFramesForStreaming.pop_back();
        buf.erase(buf.begin(),buf.end());
        cv::imencode(".jpg", currentFrame, buf);
        printf("NrBytes: %d\n", (int)buf.size());
        buf.shrink_to_fit();
        unsigned int numarPachete, numarBytes;
        numarBytes = buf.size();
        if(numarBytes % (BUFSIZE - sizeof(unsigned int)) != 0)
        {
            numarPachete = numarBytes / (BUFSIZE - sizeof(unsigned int)) + 1;
        }
        else
        {
            numarPachete = numarBytes / (BUFSIZE - sizeof(unsigned int));
        }
        std::vector<uchar> header;
        for(int i = sizeof(unsigned int) - 1 ; i >= 0 ; i--)
        {
            header.insert(header.begin(), GET_BYTE(numarBytes,i,sizeof(unsigned int)));
        }
        //adaug numarul total de bytes din imagine
        for(int i = sizeof(unsigned int)-1 ; i >= 0 ; i--)
        {
            header.insert(header.begin(), GET_BYTE(numarPachete,i,sizeof(unsigned
int)));
        }
        //adaug numarul total de pachete

        header.shrink_to_fit();
        imaginePNG = header.data();
        sendToUdpClient(imaginePNG, header.size());
        //Trimit cele doua valori clientului

        while(!headerReceived && streaming) //verific daca clientul a primit valorile
        {
            std::this_thread::sleep_for(std::chrono::milliseconds(5));
        }
        headerReceived = false;
    }
}
```

Fig. 3.1.4.2 – Codarea imaginii și transmiterea dimensiunii totale și a numărului total de pachete

Imaginile (care sunt preluate din coada *qFetch*, cea în care adaugă thread-ul **main**) sunt împărțite în pachete de câte 1024 de octeți. Fiecărui pachet i se adaugă index-ul la început, astfel încât la recepție să se poate face o ordonare a lor, dacă nu au fost recepționate în ordine, însă nu se poate face nimic în legătură cu pachetele pierdute. (Fig. 3.1.4.3)

```
for(unsigned int k = 0 ; k < numarPachete ; k++) //trimit pachete de cate BUFSIZE octeti
{
    if(k == numarPachete - 1)
    {
        end = buf.size() - 1;
        std::vector<uchar> newBuf(&buf[begin], &buf[end]);
        for(int i = sizeof(unsigned int)-1 ; i >= 0 ; i--)
        {
            newBuf.insert(newBuf.begin() , GET_BYTE(k,i,sizeof(unsigned
int)));
        }
        printf("\n=====\n");
        printf("Index: %d\n",k);
        printf("Begin: %d\n",begin);
        printf("End: %d\n",end);
        printf("Total: %d\n", (int)buf.size());
        printf("End - Begin: %d\n", end - begin + 1);
        newBuf.shrink_to_fit();
        imaginePNG = newBuf.data();
        printf("Length: %d\n", (int)newBuf.size());
        sendToUdpClient(imaginePNG,newBuf.size());
        printf("=====\n");
    }
}
```

Fig. 3.4.1.3 – Adăugarea index-ului în pachet și transmiterea lui la receptor

3.1.5. Firul de execuție Door

Scopul acestui proiect este acela de a controla accesul într-o clădire. Așa cum s-a menționat în Capitolul 2.3, a fost proiectat un driver pentru încuietoarea electromagnetă (Fig. 2.3.2). Un circuit cu doi tranzistori bipolari care să facă interfațarea între pinii de uz general ai plăcii Raspberry Pi și încuietoarea electromagnetă.

Firul de execuție **door**, este cel responsabil de controlul pinului 12 al microcontrolerului. Deoarece driver-ul funcționează în regim de switch cu logică inversată, atunci când acest pin este în starea LOW, ușa este descuiată, iar dacă este în starea HIGH, ușa este încuiată. Acest thread așteaptă o comandă de UNLOCK de la thread-ul **serverTCP**. Când primește această comandă, schimbă starea pin-ului pentru o perioadă de timp prestabilită de administratorul sistemului (implicit 10 secunde). La startul aplicației, se testează încuietoarea (se descuie pentru 5 secunde), astfel, utilizatorul va ști dacă ceva nu este în regulă.

Pentru a avea acces la pinii microcontroler-ului, comunitatea Raspberry Pi a dezvoltat o bibliotecă de funcții numită **WiringPi**. Această bibliotecă de funcții dă posibilitatea utilizatorului de a putea controla pinii de uz general ai procesorului, dar și alte funcționalități.

Fig. 3.1.5.1 reprezintă secvența de cod prin care se asigură această funcționalitate.

```
void* openDoor(void *argument)
{
    printf("Starting door thread...\n");
    if(wiringPiSetup() == -1)
    {
        printf("Could not initialize pin library!\n");
        exit(1);
    }else{
        printf("GPIO pins ready to use!\n");
    }

    pinMode(1,OUTPUT);
    printf("Testing the door...\n");
    digitalWrite(1,LOW);
    std::this_thread::sleep_for(std::chrono::milliseconds(5000));
    digitalWrite(1,HIGH);
    printf("Test ok...\n");

    while(1)
    {
        if(openFlag)
        {
            digitalWrite(1,LOW);
            std::this_thread::sleep_for(std::chrono::milliseconds(10000));
            digitalWrite(1,HIGH);
            openFlag = false;
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
    }
    return NULL;
}
```

Fig. 3.1.5.1 – Funcția de descuiere a ușii

3.1.6. Baza de date a utilizatorilor și desfășurător

Una dintre atribuțiile thread-ului **serverTCP** este aceea de a administra bazele de date cu utilizatori și log-ul cu acțiunile desfășurate în cadrul sistemului. Aceste informații sunt stocate în două fișiere XML. A fost ales acest format pentru că, așa cum s-a menționat și în Capitolul 2.4, este un format standardizat, în care informațiile pot fi structurate într-o manieră logică. Pentru acest tip de format au fost dezvoltate biblioteci de funcții care să faciliteze scrierea și citirea, astfel încât aceste operații să fie făcute cât mai eficient. Deoarece este standardizat, este și portabil, ceea ce îl face foarte potrivit pentru comunicațiile prin Internet.

La nivelul întregului sistem, utilizatorii sunt împărțiți în doua categorii:

- **Standard** – Utilizatorii standard au posibilitatea de a deschide ușa și de a urmări imaginile preluate de camera video;
- **Administrator** – În plus față de utilizatorii standard, administratorii au posibilitatea de a adăuga, șterge sau edita utilizatori (cu constrângerea că nu se pot șterge pe sine, astfel evitându-se situația în care nu ar mai exista nici un administrator în sistem) și pot urmări desfășurătorul (log-ul) cu toate acțiunile efectuate de utilizatori.

Baza de date cu utilizatori

Fișierul *users.xml*, în care pot face modificări doar administratorii, are structura următoare din Fig. 3.1.6.1:

```
<?xml version="1.0" encoding="UTF-8"?>
<users_document>
  <user>
    <first_name>Andrei</first_name>
    <last_name>Popescu</last_name>
    <username>apopescu</username>
    <password>parola</password>
    <administrator>true</administrator>
  </user>
</users_document>
```

Fig. 3.1.6.1 – Structură fișier utilizatori

Așa cum reiese din Fig. 3.1.6.1, pentru fiecare persoană înregistrată în sistem, se salvează următoarele informații:

- Prenume;
- Nume;
- Nume utilizator;
- Parola;
- Dacă persoana respectivă are drepturi de administrator (*true*) sau nu (*false*).

Baza de date cu acțiunile desfășurate de utilizatori

Acțiunile posibile în sistem sunt împărțite în cinci categorii:

- Utilizatorul s-a logat în aplicație;
- Utilizatorul a descuiat ușa;
- Utilizatorul a urmărit imaginile de la cameră;
- Administratorul a intrat în zona de editare a utilizatorilor (secțiune disponibilă în aplicația mobilă);
- Administratorul a modificat, adăugat sau șters un utilizator.

Fișierul *logs.xml* are structura din Fig. 3.1.6.2.

```
<?xml version="1.0" encoding="UTF-8"?>
<log_document>
  <action>
    <type>user_logged_in</type>
    <time>12:40:05|Mi 11 apr 2018</time>
    <first_name>Andrei</first_name>
    <last_name>Popescu</last_name>
    <username>apopescu</username>
    <administrator>true</administrator>
  </action>
</log_document>
```

Fig. 3.1.6.2 – Structura fișier desfășurator acțiuni (log)

Așa cum reiese din tabelul anterior, pentru fiecare acțiune se înregistrează următoarele informații:

- Tipul acțiunii (unul din cazurile prezentate mai sus);
- Timpul la care a avut loc (ora, ziua, luna, anul);
- Prenumele utilizatorului care a efectuat această acțiune;
- Numele;
- Numele de utilizator;
- Dacă persoana are drepturi de administrator, sau nu;

Pentru sporirea securității sistemului, în versiuni ulterioare ale programului se pot implementa algoritmi de criptare/decriptare pentru aceste fișiere. Sistemul de operare Linux poate fi securizat prin setarea unei parole *root* puternice, însă un nivel suplimentar de securitate este necesar având în vedere tipul proiectului.

3.2. Aplicația mobilă (client)

Pentru a realiza interfața cu utilizatorul, s-a ales dezvoltarea unei aplicații Android astfel încât utilizatorul să poate accesa cât mai ușor sistemul, iar operația de descuiere a ușii să se facă facil și în siguranță. Aplicația a fost dezvoltată în limbajul Java, iar mediul de dezvoltare utilizat este Android Studio 3.0.1. În cadrul acestei aplicații, un utilizator (în funcție de drepturile pe care le are) poate efectua oricare din acțiunile descrise în Capitolul 3.1.6.

Aplicația client are următoarele cerințe:

- Stabilirea conexiunii cu server-ul și menținerea acesteia până la închiderea aplicației;
- Autentificarea utilizatorului (prin introducerea parolei sau folosirea senzorului de amprentă) și stabilirea tipului acestuia;
- Modificarea meniului principal în funcție de permisiunile utilizatorului;
- Trimiterea de comenzi și procesarea răspunsurilor de la server;
- Recepționarea imaginilor de la server, decodarea și afișarea lor pe ecran;

3.2.1. Prezentare generală

La fel ca la aplicația server, pentru ca aplicația să ruleze, a fost necesară crearea de fire de execuție pentru diferitele module ale aplicației. Orice aplicație Android este împărțită implicit pe două fire de execuție. Primul thread este responsabil de interfața grafică, iar celalalt se ocupă de restul operațiilor ce nu au legătură cu interfața utilizator. Pe lângă aceste două fire implicite, au fost implementate alte două fire de execuție, pentru conexiunea TCP respectiv UDP.

La pornirea aplicației, se încearcă conexiunea la server prin TCP. Dacă conexiunea are loc cu succes, utilizatorului i se cere user-ul și parola (adăugate anterior de către un administrator). Dacă server-ul nu răspunde (este down sau telefonul nu are conexiune la internet), utilizatorul este notificat de acest lucru printr-un dialog. Așa cum s-a menționat și în Capitolul 3.1.6, utilizatorii sunt împărțiți în cele două categorii: standard și utilizatori. Această separare este vizibilă clar în cadrul aplicației mobile prin opțiunile disponibile fiecăruia, administratorul având la dispoziție două ferestre în plus față de utilizatorul standard. În Fig. 3.2.1.1 reprezintă fereastra *schema* bloc a aplicației.

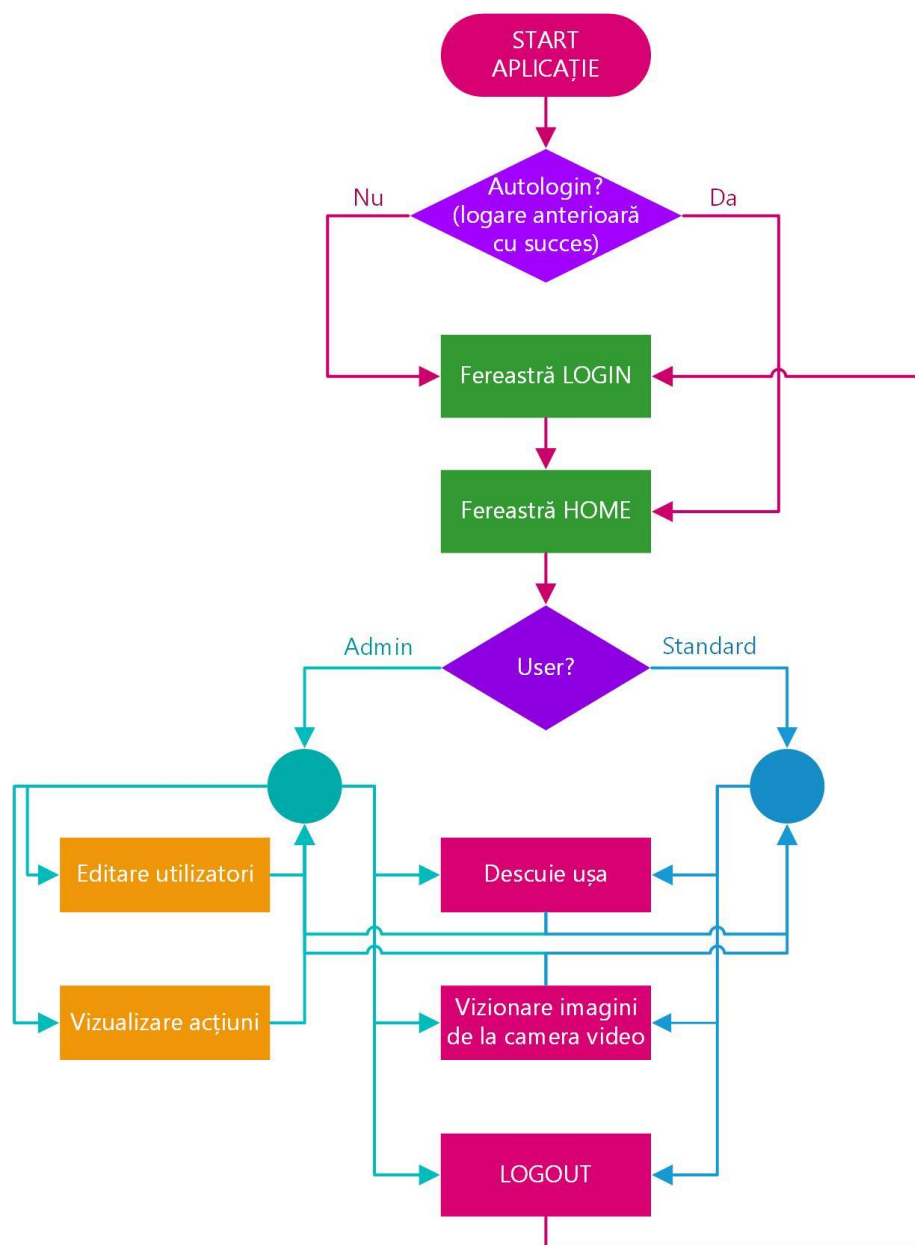


Fig. 3.2.1.1 – Schema bloc a aplicației mobile (ferestrele aplicației și acțiunile disponibile utilizatorilor)

3.2.2. Componentele aplicației mobile

Deoarece aplicația client este scrisă în limbajul Java, trebuie implementate principiile programării orientate obiect, astfel încât aceasta să fie structurată într-o manieră logică, și aplicația să ruleze eficient, cu o utilizare optimă a resurselor. Clasele principale care compun aplicația sunt:

- **User** – container pentru informațiile despre utilizator;
- **TcpClient** – clasă *Singleton* care moștenește clasa *Thread*, implementează conexiunea cu server-ul prin protocolul TCP. Această funcționalitate rulează pe un fir de execuție separat de restul aplicației. Este clasă de tip *Singleton*, deoarece există o singură cale de comunicație cu server-ul, și nu este de dorit să poată fi create alte instanțe ale acestei clase;
- **UdpClient** – clasă, de asemenea *Singleton* (din același motiv ca și clasa **TcpClient**) care moștenește clasa *Thread*, implementează conexiunea prin protocolul UDP cu server-ul. Si această funcționalitate rulează pe un fir de execuție separat;
- **TcpSubject** și **TcpObserver** – cele două clase care intră în componența modelului de proiectare *Observer*. Este folosit pentru a facilita trimiterea răspunsului de la server către toate clasele care doresc comunicația cu server-ul prin protocolul TCP;
- **UdpSubject** și **UdpObserver** – la fel ca la clasele anterioare, se implementează design pattern-ul *Observer* pentru a trimite imaginea primită de la server prin UDP către toate clasele care doresc să o afișeze;
- **Constants** – container pentru toate variabilele constante, disponibile în întreaga aplicație;
- **StartActivity** – punctul de intrare în aplicație. Aici se verifică dacă a avut loc o autentificare în trecut, și dacă a fost bifată opțiunea „Remember username and password”, se afișează direct fereastra *Home*, în caz contrar se va afișa fereastra *Login*;
- **LoginActivity** – fereastra *Login*;
- **MainActivity** – clasă care reprezintă un schelet pentru diferitele ferestre (*fragments*) ale aplicației. Această clasă administrează schimbările de context în cadrul interfeței utilizator;
- **HomeFragment** – fereastra *Home*. Aici, utilizatorul are posibilitatea de a deschide ușa și de a urmări imaginile preluate de camera video;
- **AdministratorFragment** – fereastra de editare a utilizatorilor Administratorul poate adăuga, șterge sau edita utilizatori;
- **LogsFragment** – fereastra în care administratorul are posibilitatea de a urmări toate acțiunile înregistrate în sistem. Acesta are la dispoziție o funcție de filtrare a acestora după tipul și/sau perioada de timp în care au fost efectuate;

Relațiile dintre aceste clase se pot observa în diagrama UML (*Unified Modelling Language*) din Fig. 3.2.2.1.

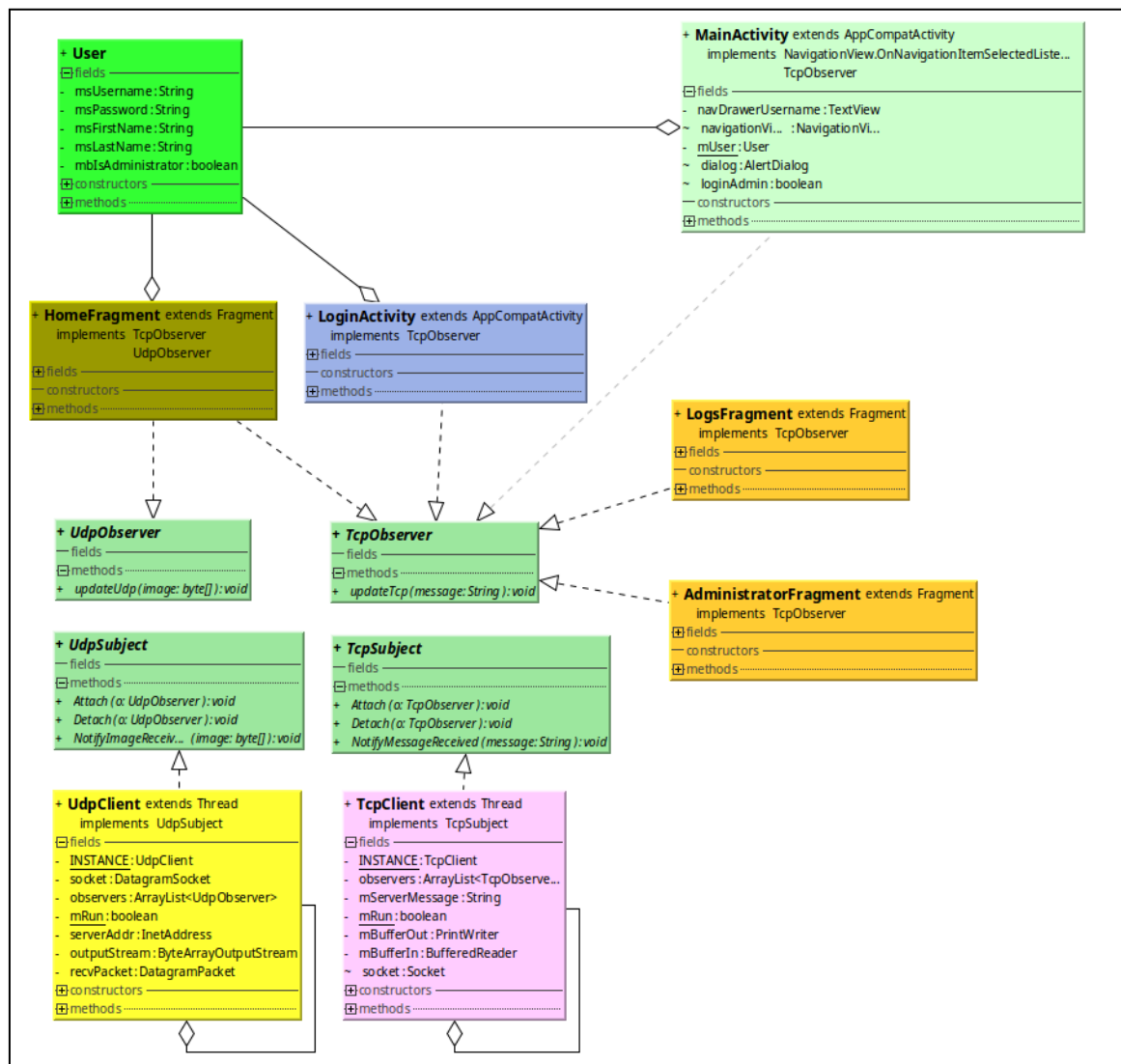


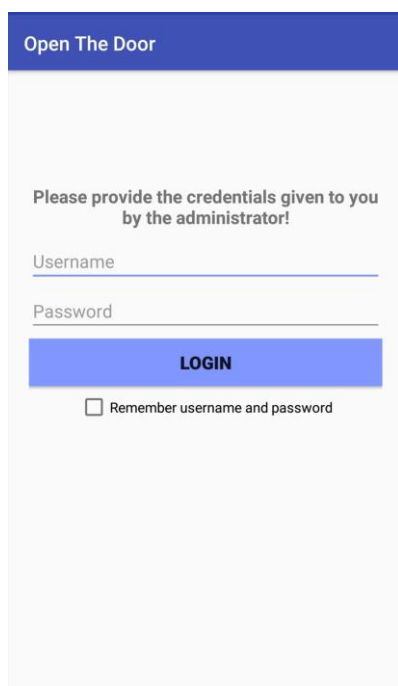
Fig. 3.2.2.1 – Diagrama UML a aplicației mobile (client)

Din diagrama din Fig 3.2.2.1 reies următoarele aspecte, prezentate și mai sus:

- Clasele **HomeFragment**, **LogsFragment**, **AdministratorFragment**, **LoginActivity** și **MainActivity** (Anexa) sunt „observatori” ai conexiunii la clientul TCP. Aceste obiecte trimit și primesc mesaje prin intermediul clasei **TcpClient** care reprezintă „subiectul” (denumiri preluate din modelul de proiectare *Observer*);
- Doar clasa **HomeFragment** este „observator” pentru **UdpClient**, deoarece doar în această fereastră vor fi afișate imaginile primite de la server;
- Clasele **UdpClient** și **TcpClient** sunt de tip *Singleton* deoarece au ca membru obiecte de tipul lor.

Așa cum se observă în Fig. 3.2.1.1, la deschiderea aplicației se verifică dacă pe telefonul respectiv a avut loc o autentificare anterioară, la care s-a bifat opțiunea „Remember username and password”. Dacă nu, atunci se deschide fereastra de Login, în care utilizatorul trebuie să se autentifice cu username și parola, după care se deschide fereastra *Home*. Dacă opțiunea a fost bifată anterior, utilizatorul nu mai trebuie să se autentifice, și aplicația se deschide automat.

Figurile 3.2.2.2 și 3.2.2.3 reprezintă fereastra *Login* respectiv *Home*.



The screenshot shows a mobile application interface for login. At the top, there is a blue header bar with the text "Open The Door". Below the header, the main area is light gray. It contains the instruction "Please provide the credentials given to you by the administrator!". There are two input fields: "Username" and "Password". Below these fields is a blue button labeled "LOGIN". At the bottom, there is a checkbox labeled "Remember username and password".

Fig. 3.2.2.2 – Login

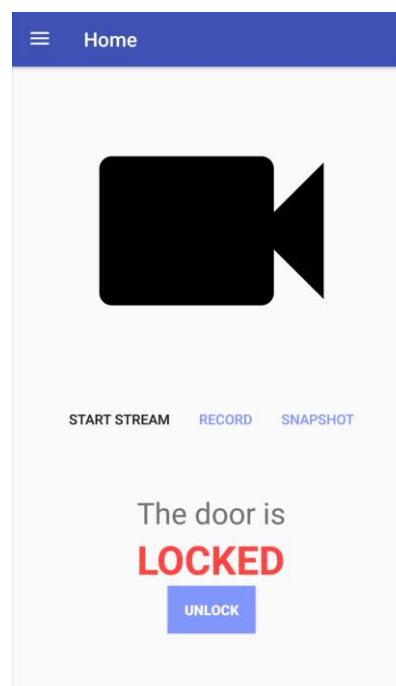


Fig. 3.2.2.3 – Home

Dacă se glisează din partea stângă a ecranului în fereastra *Home*, va apărea meniul aplicației, în care sunt disponibile toate secțiunile. Diferențierea dintre cele două tipuri de utilizatori este reprezentată în figurile Fig. 3.2.2.5 și Fig. 3.2.2.6. Dacă o persoană dorește să deschidă ușa, trebuie să apese butonul *UNLOCK* din fereastra *Home* (Fig. 3.2.2.3). Din motive de securitate utilizatorul este rugat să reintroducă parola sau să folosească senzorul de amprentă (dacă telefonul mobil dispune de un astfel de senzor). Dacă autentificarea are succes, ușa se va deschide pentru o perioadă presetată de timp. Fig. 3.2.2.4 reprezintă dialogul de autentificare pentru descurierea ușii.

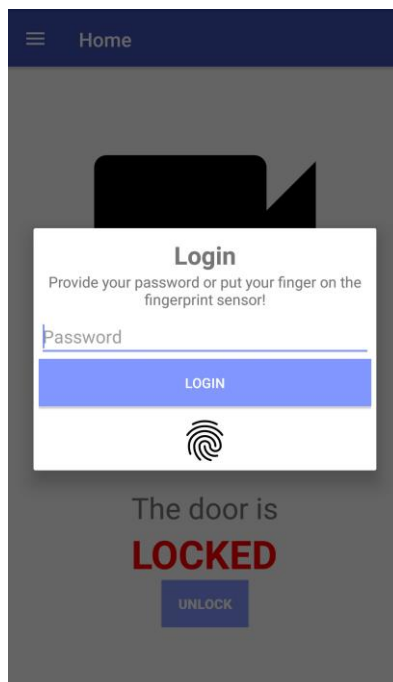


Fig. 3.2.2.4 – Dialog autentificare la descuierea ușii

Așa cum s-a menționat în capitolele anterioare, administratorul are posibilitatea de a edita baza de date cu utilizatori, și de a urmări acțiunile desfășurate în cadrul sistemului. Diferențierea dintre utilizatorii standard și administratori se vede clar în meniurile disponibile fiecărui tip. Figurile 3.2.2.5 și 3.2.2.6 reprezintă meniul utilizatorilor standard, respectiv administratorilor.

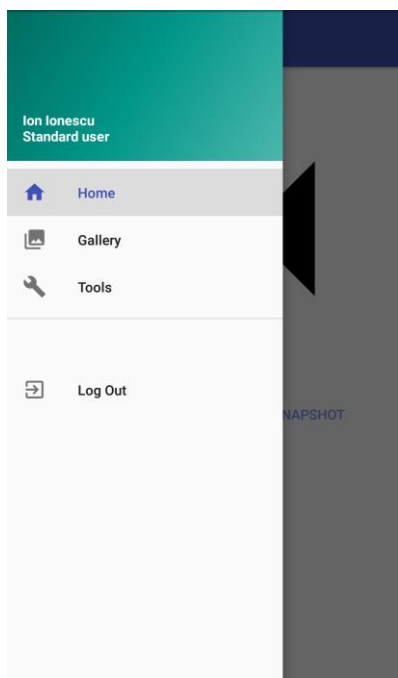


Fig. 3.2.2.5 – Meniu utilizator standard

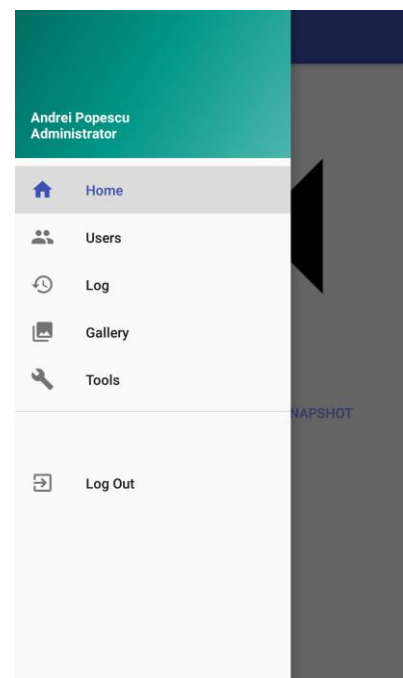
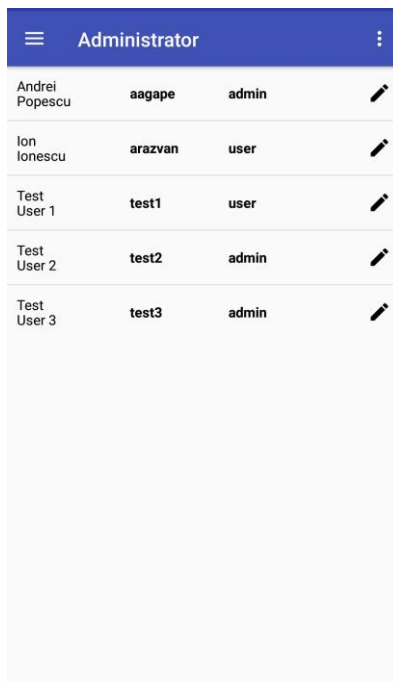


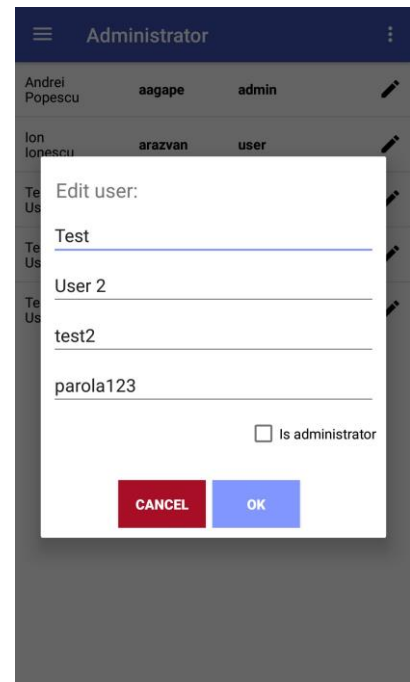
Fig. 3.2.2.6 – Meniu administrator

Secțiunea din aplicație prin intermediul căreia administratorul are acces la lista utilizatorilor este reprezentată în Fig. 3.2.2.7, dialogul de editare a unui utilizator este prezentat în Fig. 3.2.2.8, lista de acțiuni în Fig. 3.2.2.9 și dialogul de filtrare a acestora în Fig. 3.2.2.10.



Administrator			
Andrei Popescu	aagape	admin	
Ion Ionescu	arazvan	user	
Test User 1	test1	user	
Test User 2	test2	admin	
Test User 3	test3	admin	

Fig. 3.2.2.7 – Listă utilizatori



Administrator

Edit user:

Test

User 2

test2

parola123

☐ Is administrator

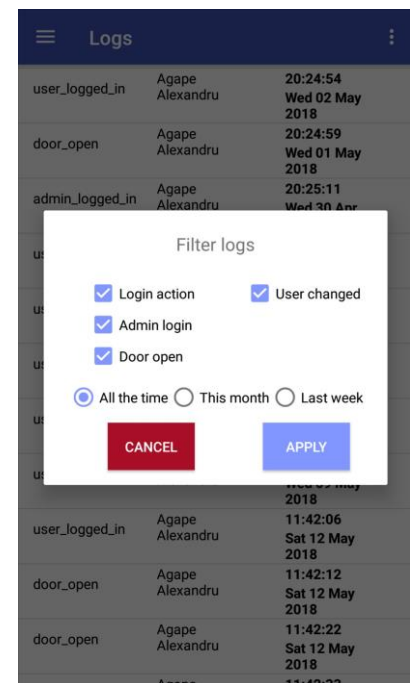
CANCEL OK

Fig. 3.2.2.8 – Editare utilizator



Logs		
user_logged_in	Agape Alexandru	20:24:54 Wed 02 May 2018
door_open	Agape Alexandru	20:24:59 Wed 01 May 2018
admin_logged_in	Agape Alexandru	20:25:11 Wed 30 Apr 2018
user_changed	Agape Alexandru	20:25:23 Wed 09 May 2018
user_logged_in	Agape Alexandru	20:25:55 Wed 09 May 2018
user_logged_in	Agape Alexandru	20:27:53 Wed 09 May 2018
user_logged_in	Agape Alexandru	20:29:54 Wed 09 May 2018
user_logged_in	Agape Alexandru	20:35:40 Wed 09 May 2018
user_logged_in	Agape Alexandru	11:42:06 Sat 12 May 2018
door_open	Agape Alexandru	11:42:12 Sat 12 May 2018
door_open	Agape Alexandru	11:42:22 Sat 12 May 2018
door_open	Agape Alexandru	11:42:33 Sat 12 May 2018

Fig. 3.2.2.9 – Listă acțiuni



Logs

Filter logs

☒ Login action ☒ User changed

☒ Admin login

☒ Door open

☒ All the time ☐ This month ☐ Last week

CANCEL APPLY

Fig. 3.2.2.10 – Filtrare acțiuni

3.2.3. Metodele de proiectare folosite (*design patterns*)

În cadrul aplicației mobile, s-au folosit două metode de proiectare (*design patterns*):

- *Singleton* – O clasă de tip *Singleton* are particularitatea că poate fi instanțiată o singură dată;
- *Observer* – O dependență *one-to-many* între obiecte, în așa fel încât atunci când un obiect își schimbă starea, toate celelalte obiecte care intră în dependență să afle acest lucru;

Singleton restricționează instanțierea unei clase. În anumite situații poate fi extrem de util ca unele clase să aibă exact un obiect. Acest lucru se implementează prin forțarea constructorului ca funcție privată și crearea (unicul apel al constructorului) unei instanțe statice ca membru al clasei, iar accesul la aceasta se face cu o funcție care returnează acel obiect. Fig. 3.2.3.1 reprezintă structura unei clase *Singleton*:

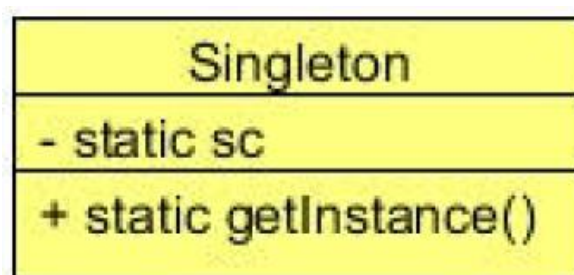


Fig. 3.2.3.1 – Structura modelului *Singleton*[19]

Se poate observa în figura anterioară obiectul static (membru al clasei) de același tip cu clasa și funcția care returnează acea instanță. Astfel, orice clasă din cadrul întregului proiect care are nevoie de o funcționalitate implementată de clasa *Singleton*, trebuie mai întâi să acceseze instanța și apoi să apeleze funcția respectivă. Acest design pattern este extrem de folosit în practică, deoarece implementează o caracteristică necesară în foarte multe situații.

Observer, așa cum s-a spus mai sus, reprezintă o dependență *one-to-many* între două sau mai multe clase, în așa fel încât, atunci când un obiect își schimbă starea, celelalte obiecte să știe acest lucru. Obiectele cheie în acest model sunt Subiectul (*Subject*) și Observatorii (*Observers*). Un subiect poate avea orice număr de observatori dependenți. O astfel de dependență este extrem de utilă în unele situații, deoarece implementează un eveniment la care pot răspunde mai multe obiecte, care pot să nu aibă nici o legătură între ele (ex. Moștenire, sau polimorfism). Acest *design pattern* se implementează prin crearea a două interfețe (Fig. 3.2.3.2). Acestea impun definirea a patru funcții:

- **Update()** – Funcție definită pentru fiecare observator și apelată de către subiect;
- **registerObserver()** – Adăugarea unui observator în listă;
- **removeObserver()** – Eliminarea unui observator din listă;
- **notifyObservers()** – Apelul fiecărei funcții *update()* corespunzătoare fiecărui observator în momentul schimbării stării subiectului.

Fig. 3.2.3.2 reprezintă principiul după care funcționează acest model.

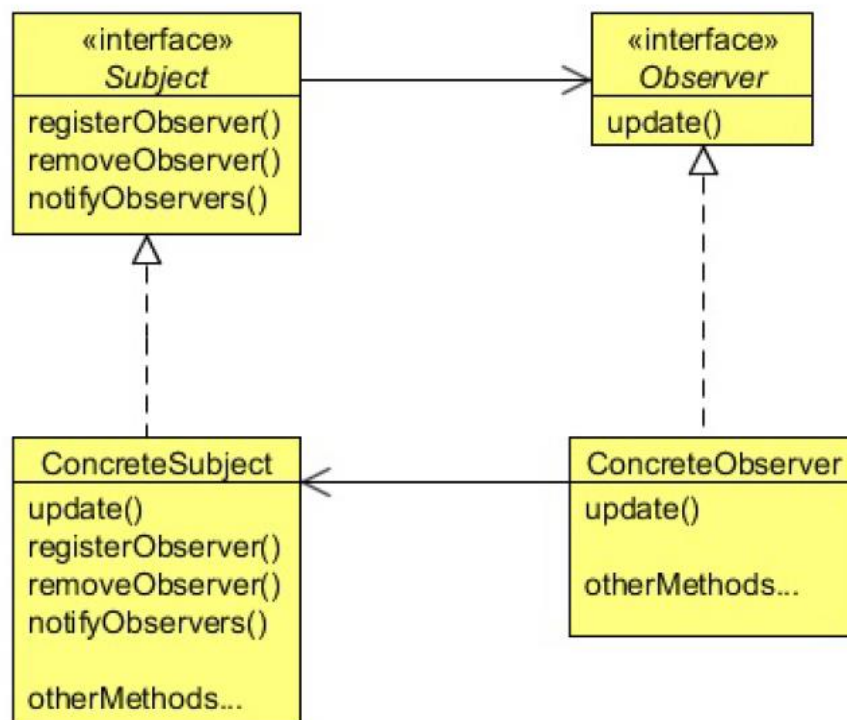


Fig. 3.2.3.2 – Principiul de funcționare al modelului *Observer* [19]

3.2.4. Clasa **TcpClient**

Una din cerințele aplicației mobile este aceea de a stabili și de a menține o conexiune cu server-ul, folosind protocolul TCP, pentru transmiterea de comenzi și procesarea răspunsurilor de la acesta. Așa cum s-a menționat pe scurt în secțiunile anterioare, clasa **TcpClient** a fost implementată după modelul *Singleton* (Fig. 3.2.4.1). Acest lucru este extrem de util în acest caz, deoarece există o singură conexiune (*socket*) cu server-ul la nivelul întregii aplicații. Instanțierea unui nou obiect de tipul acestei clase ar însemna deschiderea unei noi conexiuni, ceea ce se traduce printr-o încărcare inutilă a server-ului, deoarece acesta percepe orice nouă conexiune ca fiind un nou client și creează un alt thread care să administreze cererile de la clientul respectiv.

Această clasă moștenește clasa *Thread* din biblioteca standard Java, fapt ce indică faptul că funcționalitățile implementate de această clasă vor rula pe un fir de execuție separat de restul aplicației (Fig. 3.2.4.2). Pentru a da posibilitatea și altor clase să comunice cu server-ul TCP de pe placa Raspberry Pi, a fost implementat și design pattern-ul *Observer*. Acesta este format din două interfețe (*Subject* și *Observer*). Astfel, clasa **TcpClient** este *Subject*, deoarece atunci când server-ul trimite un răspuns, acesta să poată ajunge la toate clasele interesate (*Observers*) de răspunsul respectiv. (Fig. 3.2.4.3).

```
public class TcpClient extends Thread
    implements TcpSubject {

    private static TcpClient INSTANCE = new TcpClient(); //Singurul obiect creat de tipul
        acestei clase, static

    //Clasa Singleton pentru o singura instantiere a TCPClient

    private ArrayList<TcpObserver> observers;//lista de observatori
    private String mServerMessage; //Mesajul primit de la server
    private static boolean mRun = false; //variabilă care spune daca clientul este activ
    private PrintWriter mBufferOut; //obiect care se ocupa de transmiterea mesajelor
    private BufferedReader mBufferIn; // obiect care se ocupa de receptionarea mesajelor
    Socket socket;// Conexiunea la server

    //=====
    private TcpClient(){ //Constructor privat pentru a forța crearea unei singure instante
        observers = new ArrayList<TcpObserver>();
        mServerMessage = "";
    }
    //=====
    public static TcpClient getInstance(){
        return INSTANCE;
    }//Functie static, care returneaza obiectul, pentru ca celelalte clase sa aibă acces
    la funcția de trimitere mesaj. (Anexa)
```

Fig. 3.2.4.1 – Clasa **TcpClient**, implementarea modelului de proiectare *Singleton*

```

public void run(){

    mRun = true;

    try{
        InetAddress serverAddr = InetAddress.getByName(Constants.SERVER_ADDRESS);
        Log.d("TcpClient ", " Connecting...");
        socket = new Socket(serverAddr, Constants.TCP_PORT); // Se stabileste conexiunea
        try{
            mBufferIn = new BufferedReader(new
                InputStreamReader(socket.getInputStream()));

            int charsRead = 0;
            char[] buffer = new char[1024];
            NotifyMessageReceived(Constants.CONNECTION_ESTABLISHED);

            while (mRun){ //Menținerea conexiunii până când aplicația este oprită

                charsRead = mBufferIn.read(buffer); // Se asteapta mesaj, sau o
                //din acesta
                mServerMessage = mServerMessage + new String(buffer).substring(0,
                charsRead); // Se adauga porțiunea citita la restul mesajului

                if(mServerMessage.contains(Constants.MESSAGE_STOP_FLAG)) {
                    //Dacă s-a identificat finalul de mesaj, acesta se șterge
                    int flagPos = mServerMessage.indexOf(Constants.MESSAGE_STOP_FLAG) - 1;
                    mServerMessage = mServerMessage.substring(0, flagPos);

                    Utils.runOnUiThread(new Runnable() {
                        @Override
                        public void run() {
                            NotifyMessageReceived(mServerMessage); //Notificare msj. primit
                        }
                    });
                }

                Log.e("RESPONSE FROM SERVER: ", " Received message "+mServerMessage);
            } catch (Exception e){
                Log.e("Error! ", "Error at creating buffers!");
                stopClient();
                e.printStackTrace();
            }
            finally {
                Log.d("Finally", "Socket closed!");
                stopClient();
                NotifyMessageReceived(Constants.CONNECTION_LOST);
            }
        } catch (Exception e){
            Log.e("Error! ", "Error at connecting to the server!");
            stopClient();
            NotifyMessageReceived(Constants.SERVER_DOWN);
            e.printStackTrace();
        }
    }
}

```

Fig. 3.2.4.2 – Clasa **TcpClient**, menținerea conexiunii, primirea mesajelor și notificarea acestui fapt către observatori

Tabelul anterior arată și faptul că, în eventualitatea în care aplicația server nu mai răspunde, utilizatorul este notificat de acest lucru printr-un mesaj. Acest mesaj este transmis către toate ferestrele aplicației. Utilizatorul are posibilitatea de a restabili conexiunea prin intermediul unui buton poziționat în dialogul respectiv.

Mesajele recepționate conțin și un flag de stop, astfel încât clientul să știe atunci când a primit tot mesajul. Dacă flag-ul a fost identificat în cadrul mesajului, **TcpClient** îl trimite mai departe observatorilor prin apelul funcției *NotifyMessageReceived(mesaj)*.

Fig. 3.2.4.3 reprezintă interfețele *Subject* și *Observer*, care implementează modelul de Proiectare *Observer*.

```
public interface TcpObserver {
    void updateTcp(String message);
}
//=====
public interface TcpSubject {
    void Attach(TcpObserver o);
    void Detach(TcpObserver o);
    void NotifyMessageReceived(String message);
}
```

Fig 3.2.4.3 – Interfețele **TcpObserver** și **TcpSubject**

Funcțiile din Tabelul anterior au următoarele scopuri:

- **updateTcp** – Această funcție este apelată de fiecare dată când apare un nou mesaj de la server;
- **Attach** – Prin apelul acestei funcții, o clasă se poate „înscris” în lista de observatori pentru clientul TCP;
- **Detach** – Prin apelul funcției *Detach*, un observator poate ieși din listă;
- **NotifyMessageReceived** – Această funcție este apelată de către *Subject* pentru a notifica observatorii de recepționarea unui nou mesaj.

3.2.5. Clasa UdpClient

Altă cerință a aplicației este aceea de a primi și afișa imaginile de la camera video (pe care server-ul le trimite prin protocolul UDP). Dacă server-ul trimite aceste imagini prin UDP, este nevoie de o aplicație client care să primească aceste imagini și să le proceseze. Clasa **UdpClient** a fost implementată într-o manieră similară cu **TcpClient**, ea fiind de asemenea de tip *Singleton*. La fel ca și la clasa prezentată în capitolul anterior, a fost folosit modelul *Observer* pentru a semnala recepționarea cu succes a unei imagini.

Aplicația server, pentru a reduce considerabil dimensiunea imaginii, o compresează și o codează în formatul JPEG. Principala dificultate care a fost întâmpinată la implementarea acestei clase a fost decodarea imaginii. S-a încercat inițial decodarea folosind o funcție din clasa *Bitmap*, care aparține bibliotecii Android. Din motive necunoscute acea funcție a returnat în repetate rânduri cod de eroare, și s-a renunțat la ea. Următoarea abordare a fost să se implementeze decodarea folosind biblioteca OpenCV (cea folosită de aplicația server) și s-a ajuns într-un nou impas cauzat de dificultatea instalării acestei biblioteci în mediul de dezvoltare Android Studio.

Fig. 3.2.5.1 prezintă membrii clasei și arată faptul că este clasă de tip *Singleton* (instanță statică ca și membru al clasei, constructorul privat și funcție care returnează instanța respectivă).

```
public class UdpClient extends Thread implements UdpSubject {
    private static UdpClient INSTANCE = new UdpClient();

    private DatagramSocket socket;
    private ArrayList<UdpObserver> observers;
    private static boolean mRun = false;
    private InetAddress serverAddr;
    private ByteArrayOutputStream outputStream = new ByteArrayOutputStream( );
    private DatagramPacket recvPacket;

    //Singleton class for a single instance of UDP Client
    private UdpClient() {
        observers = new ArrayList<>();
    }

    public static UdpClient getInstance(){
        return INSTANCE;
    }
}
```

Fig. 3.2.5.1 – Implementarea clasei **UdpClient**. Membrii clasei și componentele modelului *Singleton*

Similar cu **TcpClient**, clasa **UdpClient** moștenește clasa **Thread** din biblioteca standard Java, ceea ce înseamnă că această funcționalitate va rula pe un fir de execuție separat. Înainte de a recepționa efectiv imaginea, clientul așteaptă dimensiunea totală a acesteia și numărul de pachete în care a fost împărțită. După ce aceste informații au fost primite cu succes, se alocă memorie pentru imagine și se trimite confirmarea către server.

Este necesar acest pas pentru a sincroniza clientul cu server-ul la nivel de imagine. Ulterior, se primesc pachetele pe rând. Așa cum s-a menționat în Capitolul 3.1.4, fiecare pachet conține și indexul corespunzător. Clientul citește acest index, îl elimină din pachet și adaugă biții de date în locația de memorie alocată (*ByteArray* – șir de octeți). După ce se primește ultimul pachet, imaginea este decodificată și transmisă tuturor observatorilor (în cazul acestei aplicații este vorba doar de fereastra *Home*). Codul aferent celor menționate anterior poate fi găsit în Anexă.

Cele doua interfețe care compun modelul *Observer* pentru clientul UDP sunt ***UdpSubject*** și ***UdpObserver***. Implementarea este similară cu cea de la TCP, cu diferența că în acest caz mesajul nu mai este un șir de caractere, ci *ByteArray* (Fig. 3.2.5.2).

```
public interface UdpObserver {
    void updateUdp(byte[] image);
}
//=====
public interface UdpSubject {
    void Attach(UdpObserver o);
    void Detach(UdpObserver o);
    void NotifyImageReceived(byte[] image);
}
//=====
public void NotifyImageReceived(byte[] image) {
    for(int i = 0 ; i < observers.size() ; i++) {
        observers.get(i).updateUdp(image);
    }
}
//=====

public void updateUdp(final byte[] image) {
    final Bitmap bmp = BitmapFactory.decodeByteArray(image, 0, image.length);

    Utils.runOnUiThread(new Runnable() {
        @Override
        public void run() {
            imageView.setImageBitmap(bmp);
        }
    });
}
```

Fig. 3.2.5.2 - Interfețele care implementează modelul Observer pentru clasa *UdpClient*

Se poate observa, în secvența de cod anterioară, faptul că afișarea imaginii este efectuată de către thread-ul UI (apelul funcției ***runOnUiThread***).

3.2.6. Listele cu utilizatori și acțiuni

În Capitolul 3.2.2, s-au prezentat ferestrele prin intermediul cărora administratorul are acces la editarea informațiilor despre utilizatori și la vizualizarea acțiunilor pe care aceștia le-au efectuat în cadrul sistemului. Acele ferestre sunt, de fapt, două obiecte de tip *ListView* din biblioteca Android, care sunt populate cu datele corespunzătoare. Pentru ca datele să fie independente de interfața grafică, este necesar să se utilizeze un nou model de proiectare. Este de dorit ca datele să fie complet separate de interfața grafică pentru că thread-ul responsabil de actualizarea interfeței utilizator nu trebuie să aducă modificări asupra informațiilor

Pentru implementarea acestora s-a folosit un alt *design pattern* specific programării interfețelor grafice: *Model-View-Controller*. Acest pattern reprezintă, de fapt, o modalitate de a modulariza codul, astfel încât fiecare secțiune de cod să îndeplinească o anumită funcție independent de celelalte secțiuni. Cele trei componente principale ale acestui pattern sunt:

- **Model** – Modelul reprezintă un container pentru informațiile de afișat. El stabilește ce reprezintă informațiile (în cazul acestei aplicații modelul poate reprezenta clasa *User*, care definește informațiile atribuite unui utilizator: nume, prenume, etc.);
- **View** – Reprezintă componenta din interfața grafică în care vor fi prezentate administratorului informațiile (în acest caz, o listă);
- **Controller** – Controllerul face legătura între model și view. Acesta controlează fluxul de date în model și reactualizează view-ul ori de câte ori datele se modifică. Controllerul este cel care asigură separația dintre model (container-ul datelor) și view (interfața grafică).

Clasa ***AdministratorFragment*** reprezintă zona de editare a informațiilor despre utilizatori. Din motive de securitate, atunci când administratorul dorește să acceseze această secțiune a aplicației, este rugat să reintroducă numele de utilizator și parola. După ce autentificare are loc cu succes, această clasă transmite comanda server-ului pentru a trimite conținutul fișierului xml cu utilizatori.

Modelul în acest caz este reprezentat de clasa ***User***, care definește componentele unui utilizator, precum nume, prenume etc. (Cap. 3.1.6), iar controllerul este o clasă numită ***UsersCustomAdapter***, ce este prezentată în Anexă.

Această fereastră este formată din meniul accesibil în partea dreapta-sus (Fig. 3.2.2.7). Acel meniu dă posibilitatea adăugării unui utilizator nou. În Fig. 3.2.2.7 se mai poate observa și faptul că în dreptul fiecărei linii din listă, se află un buton de editare. Prin acționarea acelui buton, se deschide dialogul de editare, în care administratorul poate modifica orice informație cu privire la respectivul utilizator (Fig. 3.2.2.8). Pentru ștergerea unui utilizator se apasă lung pe linia corespunzătoare, după care administratorul este întrebat printr-un dialog dacă este sigur că dorește eliminarea acelui utilizator. Nu este posibil ca administratorul să își elimine propriul utilizator.

Clasa ***LogsFragment*** reprezintă zona de vizualizare a acțiunilor înregistrate de sistem. La fel ca la fereastra anterioară, administratorul trebuie să furnizeze numele de utilizator și parola dacă dorește să acceseze aceste informații. În acest caz, administratorul nu mai poate șterge acțiuni din listă, ele pot fi doar vizualizate și, eventual, filtrate. Modelul în acest caz este reprezentat de clasa ***Action***, care definește componentele unei acțiuni, precum tipul ei, momentul de timp în care a fost efectuată, etc. (Cap. 3.1.6), iar controlerul este clasa ***LogsCustomAdapter*** care va fi, de asemenea, prezentată în Anexă.

3.2.7. Autentificarea biometrică cu amprentă

Sistemele convenționale de securitate erau bazate fie pe cunoaștere (precum cele pe bază de parolă sau PIN) fie pe cartelă sau cheie, și erau predispuse la eșec, deoarece parola poate fi ușor uitată, iar cartela sau cheia pot fi pierdute, duplicate sau chiar furate. Pentru a adresa nevoia unui sistem de autentificare robust, sigur și foarte greu de fraudat, au fost dezvoltate metode biometrice de identificare personală.

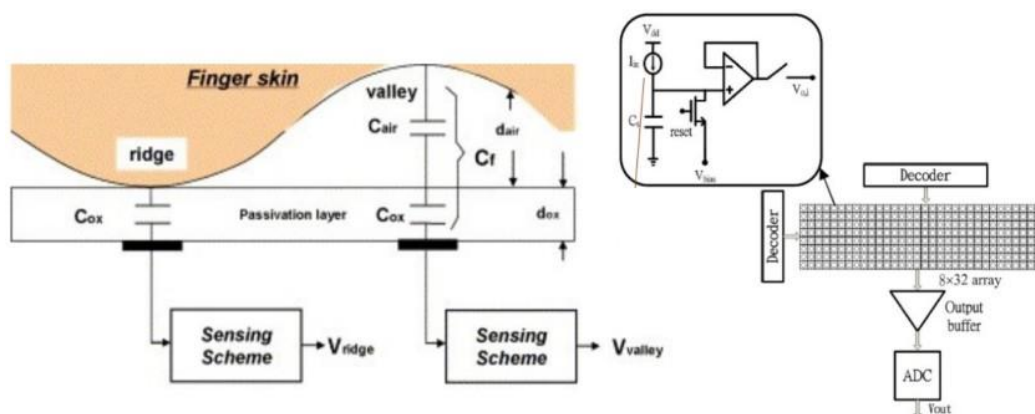
Termenul „biometric” provine din limba Greacă, și derivă din cuvintele *bio* (viață) și *metric* (a măsura). Sistemele biometrice folosesc *caracteristici fizice* ale unei persoane (precum amprenta, irisul), sau *caracteristici comportamentale* (precum vocea, scrisul de mână sau ritmul scrisului) pentru a-i determina identitatea. Un mare avantaj al sistemelor biometrice este faptul ca datele biometrice sunt unice pentru fiecare individ, ușor de obținut prin metode neintrusive și invariante în timp (nu survin modificări majore pe o perioada de timp). [20]

În cadrul acestui sistem a fost folosită identificarea prin amprentă. Amprenta, sau relief papilar se formează în perioada intrauterină a fătului și este alcătuită din creste și șanțuri care formează desenul papilar. În urma cercetărilor, antropologul englez *Francis Galton* a demonstrat că desenele papilare sunt strict individuale fiecărei, și nu pot fi găsite amprente identice nici măcar la aceeași persoană.

Procesul de identificare implică constă, în principiu, în doua etape: achiziționarea amprente și compararea acesteia cu amprente stocate în baza de date a sistemului. Achiziția amprente se face cu ajutorul senzorului de amprentă. În practică exista mai multe tipuri de senzori care funcționează pe principii diferite:

- **Senzori optici:** acest tip de senzor este prima metodă de capturare a amprentelor și se bazează pe existența unui senzor optic care fotografiază desenul papilar. Această metoda este însă ușor de fraudat deoarece preia o imagine 2D a amprente, așa că pot fi folosite imagini de o calitate mare pe care sistemul să le identifice cu succes;
 - **Senzor ultrasonic:** ultima tehnologie dezvoltată în acest domeniu, folosește un transmițător și un receptor ultrasonic. În momentul în care amprenta este așezată pe suprafața senzorului, o parte din unde sunt absorbite de deget, iar restul sunt reflectate spre receptor, în funcție de crestele și șanțurile respectivei amprente;
 - **Senzori capacitivi:** Cel mai comun tip de senzor de amprentă (folosiți preponderent la telefoanele mobile) este senzorul capacitiv. În loc să se folosească o imagine a amprente, senzorul capacitiv folosește un șir de circuite capacitive foarte mici care colectează date despre amprentă. Inițial, fiecare condensator are aceeași sarcină stocată. Atunci când degetul este așezat pe senzor, sarcina respectivă se schimbă puțin în contact cu pielea (la creste), și rămâne aproape neschimbată în zonele șanțurilor.
- Fig. 3.2.7.1 arată acest principiu. [21]

Fig.



3.2.7.1 – Principiul de funcționare al senzorului de amprentă capacitiv [21]

Cea de-a doua etapă a identificării amprentei constă în comparația amprentei achiziționate cu cea aflată în baza de date a telefonului. Pentru a avea acces la această funcționalitate s-a folosit biblioteca Android, care a pus la dispoziție funcții de verificare a amprentei. Acest lucru presupune ca utilizatorul să fi preînregistrat o amprentă în sistemul de operare. Baza de date cu amprente este deci stocată în sistemul de operare, asigurând și securitatea acestora. Algoritmul de comparare este cel folosit de sistemul de operare Android.

Clasa **FingerprintHandler**, prezentată în Tabelul 3.2.7.2, este cea responsabilă de verificarea amprentei, și de transmiterea către server a faptului că utilizatorul a fost identificat cu succes, nemaifiind necesară parola. Ea moștenește clasa **FingerprintManager.AuthenticationCallback** din Android 6.1, și are următoarele funcții implementate:

- **startAuth** – Această funcție este responsabilă de pornirea procesului de identificare a amprentei;
- **onAuthenticationError** – Este funcția apelată atunci când apare o eroare în sistem (senzorul nu răspunde sau alte erori);
- **onAuthenticationHelp** – Funcție care furnizează utilizatorului informații suplimentare despre eroarea care a apărut.
- **onAuthenticationFailed** – Funcție apelată atunci când amprenta nu este recunoscută;
- **onAuthenticationSucceeded** – Amprenta este recunoscută și este anunțată aplicația. Se afișează pe ecran faptul că autentificarea a reușit.

```

public class FingerprintHandler extends FingerprintManager.AuthenticationCallback {
    private CancellationSignal cancellationSignal;
    private Context context;
    private OnFingerprintEvent mFingerprintListener;

    public FingerprintHandler(Context mContext, OnFingerprintEvent listener) {
        context = mContext;
        mFingerprintListener = listener;
    }

    public void startAuth(FingerprintManager manager, FingerprintManager.CryptoObject
cryptoObject) {

        cancellationSignal = new CancellationSignal();
        if (ActivityCompat.checkSelfPermission(context, Manifest.permission.USE_FINGERPRINT)
!= PackageManager.PERMISSION_GRANTED) {
            return;
        }
        manager.authenticate(cryptoObject, cancellationSignal, 0, this, null);
    }

    //onAuthenticationError is called when a fatal error has occurred. It provides the error
code and error message as its parameters//

    public void onAuthenticationError(int errMsgId, CharSequence errString) {

        //I'm going to display the results of fingerprint authentication as a series of
toasts.
        //Here, I'm creating the message that'll be displayed if an error occurs//

        Toast.makeText(context, "Authentication error\n" + errString,
Toast.LENGTH_LONG).show();
        mFingerprintListener.onAuthenticationEventOccured("error");
    }

    //onAuthenticationFailed is called when the fingerprint doesn't match with any of the
fingerprints registered on the device//

    public void onAuthenticationFailed() {
        Toast.makeText(context, "Authentication failed", Toast.LENGTH_LONG).show();
        mFingerprintListener.onAuthenticationEventOccured(Constants.FINGERPRINT_AUTH_FAILED);
    }

    @Override
    public void onAuthenticationHelp(int helpMsgId, CharSequence helpString) {
        Toast.makeText(context, "Authentication help\n" + helpString,
Toast.LENGTH_LONG).show();
        mFingerprintListener.onAuthenticationEventOccured("help");
    }

    //onAuthenticationSucceeded is called when a fingerprint has been successfully matched to
one of the fingerprints stored on the user's device//
    public void onAuthenticationSucceeded(
        FingerprintManager.AuthenticationResult result) {

        Toast.makeText(context, "Authentication succeeded!", Toast.LENGTH_LONG).show();
        mFingerprintListener.onAuthenticationEventOccured(Constants.FINGERPRINT_AUTH_SUCCESS);
    }

    public interface OnFingerprintEvent{
        void onAuthenticationEventOccured(String eventName);
    }
}

```

Fig. 3.2.7.2 – Clase *FingerprintHandler*

4. Rezultate și concluzii

Ambele componente ale sistemului încorporat de securitate și control al accesului au fost testate în rețeaua de Internet locală a clădirii. Securitatea sistemului este dată de însăși rețeaua la care sunt conectate atât telefonul mobil cât și placa Raspberry Pi3. Utilizatorul trebuie să fie conectat la rețeaua WiFi astfel încât să poată accesa server-ul și să poată trimite cerere de deschidere a ușii. Numai utilizatorii deja înregistrați, care au aplicația instalată au dreptul de a intra în interiorul clădirii. Server-ul TCP a fost implementat pe portul 3600, iar cel UDP pe portul 3603. Fig. 4.1 prezintă întregul sistem cu toate componentele prezentate în capitolele anterioare.



Fig. 4.1 – Sistemul încorporat. Instalație experimentală

În Fig. 4.1 se poate observa pe monitor și detecția fețelor. Un ciclu al aplicației server (care reprezintă preluarea unei imagini, procesarea și afișarea ei) durează aproximativ 0,3 secunde, neluând în calcul eventualele cereri de la clienți, care sunt evenimente aperiodice și rare, în comparație cu procesarea imaginilor. O problemă cauzată de detecția fețelor este faptul că algoritmul folosit solicită foarte mult procesorul plăcii Raspberry Pi, acesta fiind solicitat în proporție de 50% doar de detecție. Din cercetările efectuate asupra funcției de detecție din OpenCV, s-a constatat faptul că aceasta nu are suport pentru mai multe fire de execuție.

Aplicația server consumă constant aproximativ 27 MB de memorie RAM. În testele efectuate, s-au conectat până la 4 telefoane mobile simultan, fără a se sesiza întârzieri la nivelul sistemului.

Aplicația mobilă rulează pe sisteme de operare Android 6.1 sau versiuni mai recente. Așa cum s-a spus și în Cap. 3.2.5, nu s-a reușit decodarea imaginilor din formatul JPEG în format binar pentru a putea fi afișate pe ecranul telefonului. Transmiterea efectivă a pachetelor se efectuează cu succes, lucru care reiese și din multiplele sesiuni de depanare a programului, singura problema survenind dintr-o incompatibilitate între bibliotecile OpenCV și Java/Android.

În cadrul acestui proiect s-a implementat un sistem de securitate și control al accesului cu comunicație prin Internet și aplicație mobilă, folosind o placă mini PC Raspberry Pi3. În comparație cu alte soluții discutate în literatură, sistemul propus are avantajul de a fi destul de flexibil și mai puțin expus intrușilor neautorizați, deoarece comunicația are loc prin rețeaua locală a clădirii. Prin urmare, se poate spune despre acest sistem că este la fel de sigur ca și rețeaua de Internet la care este conectat.

Însă, acest lucru poate reprezenta și o limitare, deoarece, IP-ul plăcii Raspberry Pi nu trebuie să fie făcut public. Așadar, dacă în versiunile ulterioare ale sistemului se va dori accesarea server-ului de la distanță (nu din rețeaua locală), trebuie implementați algoritmi de criptare, pentru a se asigura securitatea în comunicație.

Un alt dezavantaj al acestei implementări este reprezentat de funcția de detecție a fețelor, care suprasolicită procesorul din cauza faptului că nu suportă mai multe fire de execuție. O îmbunătățire posibilă ar putea fi implementarea algoritmului Viola-Jones cu suport pentru mai multe fire de execuție.

De-a lungul dezvoltării acestui proiect s-au întâmpinat anumite dificultăți cauzate în principal de sincronizarea între client și server, mai ales la protocolul UDP. Un alt lucru care a ridicat dificultăți a fost înțelegerea bibliotecilor Android și OpenCV. S-au petrecut multe zile de documentare pentru alegerea funcțiilor necesare și includerea acestora în cadrul proiectului.

Un avantaj al acestei implementări este reprezentat de prețul relativ scăzut al componentelor, și faptul că sunt ușor disponibile. De asemenea, Raspberry Pi are resurse mai mult decât suficiente de a rula aplicația, însemnând faptul că pot fi implementate și alte aplicații care să contribuie la caracterul inteligent al clădirii (ex. Controlul temperaturii sau umidității aerului etc.). Faptul că placa rulează un sistem de operare Linux, s-a dovedit extrem de util deoarece a dat posibilitatea programării aplicației într-un limbaj orientat obiect (C++), mărin­d flexibilitatea acesteia, și, de asemenea, a fost posibilă dezvoltarea și depanarea pe un PC care rulează un sistem de operare asemănător.

S-a încercat structurarea celor doua aplicații într-o manieră cât mai simplă, dar care să respecte anumite modele de design de bază, precum *Singleton*, *Observer* sau *Model – View – Cotroller*.

În concluzie, sistemul propus rezolvă cu succes cerințele de flexibilitate (posibilitatea de a conecta mai mulți utilizatori în același timp) și de control al accesului, însă, securitatea este dată de rețeaua la care sunt conectate cele doua terminale. În versiunile viitoare ale aplicației, toate aceste probleme pot fi rezolvate prin adăugarea unor algoritmi de criptare/decriptare.

Bibliografie

- [1] D. Hendricks (2014, April 22), *The history of Smart Homes*, URL: www.iotevolutionworld.com
- [2] S. Sruthy, G. N. Sudhish, "WiFi Enabled Home Security Surveillance System using Raspberry Pi and IoT Module", Proceedings of IEEE International Conference On Signal Processing, Informatics, Communication And Energy Systems (SPICES), Kollam, 2017
- [3] G. Sowjanya, S. Nagaraju, "Design and Implementation of Door Access Control and Security System Based on IoT", Proceedings of International Conference on Inventive Computation Technologies (ICICT), pp. 83-86, Coimbatore, 2016
- [4] Q. Al-Shebani, P. Premaratne, P. Vial, "Embedded Door Access Control Systems Based on Face Recognition: A Survey", Proceedings of the 7th International Conference on Signal Processing and Communication Systems (ICSPCS), Gold Coast, 2013
- [5] M. Sahani, C. Nanda, A. K. Sahu, B. Pattnaik, "Web-Based Online Embedded Door Access Control and Home Security System Based on Face Recognition", Proceeding of International Conference on Circuit, Power and Computing Technologies (ICCPCT), Nagercoil, 2015
- [6] W. Shao, F. D. Salim, T. Nguyen, M. Youssef, "Who Opened the Room? Device-free Person Identification Using Bluetooth Signals in Door Access" Proceeding of IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData), Exeter, pp. 68-75, 2017
- [7] I. A. Khromov, A. A. Dvornikov, "Wireless Access Monitoring and Control System Based on Intrabody Communication", Proceedings of IEEE Conference on Quality Management, Transport and Information Security, Information Technologies (IT&MQ&IS), Nalchik, 2016
- [8] S. Shavi, "Secured Room Access Module", Proceedings of International Conference On Smart Technologies For Smart Nation (SmartTechCon), Bangalore, pp.1134-1138, 2017
- [9] A. I. Lita, D. A. Visan, A. G. Mazare, L. M. Ionescu, "Door Automation System for Smart Home Implementation", Proceedings of 23rd IEEE International Symposium for Design and Technology in Electronic Packaging (SIITME), Constanta, pp. 357-360, 2017
- [10] N. A. Hussein, I. A. Mansoori, "Smart Door System for Home Security Using Raspberry Pi3", International Conference on Computer and Applications (ICCA), pp. 395-399, 2017
- [11] D. Molloy, *Exploring Raspberry Pi: Interfacing to the Real World with Embedded Linux*, Wiley, Indianapolis, 2016
- [12] D. L. Baggio, S. Emami, D. M. Escrive, K. Ievgen, J. Saragih, R. Shilkrot, *Mastering OpenCV 3*, 2nd Edition, Birmingham, 2017
- [13] Prof. Dr. Ing. Corneliu Lazăr, *Curs Sisteme de Vedere Artificială*, Universitatea Tehnică „Gh. Asachi” din Iași, Facultatea de Automatică și Calculatoare
- [14] Wikipedia contributors. (2018, May 31). XML. In *Wikipedia, The Free Encyclopedia*. Retrieved 16:27, June 8, 2018, from <https://en.wikipedia.org/w/index.php?title=XML&oldid=843767549>
- [15] K. Jamsa, L. Klander, *Totul despre C și C++, Manual fundamental de programare în C și C++*, Teora, pp. 1120 – 1140, 2001
- [16] M. Stancu, *Curs Sisteme de Operare, Servicii de Rețea*, Universitatea din Craiova, Facultatea de Științe, Departamentul de Informatică
- [17] B.A. Forouzan, *TCP/IP Protocol Suite, Fourth Edition*, McGraw-Hill, 2010
- [18] J. Chu, *Introducing Streaming Media protocols, UDP protocol illustration*, URL: <http://www.classes.dma.ucla.edu>
- [19] R. Joshi, *Java design patterns. Reusable solutions to common problems*, Exelixis Media, 2015
- [20] M. Lourde, D. Khosla, *Fingerprint identification in Biometric Security Systems*, International Journal of Computer and Electrical Engineering, Vol. 2, No. 5, October 2010
- [21] R. Triggs (2018, February 9), *How fingerprints scanners work: optical, capacitive and ultrasonic variants explained*, URL: <http://www.androidauthority.com>

Anexa 1 – Cod sursă aplicația server

main.c

```

int main(int argc, char** argv)
{
    //      pthread_t ThreadServer;
    pthread_t ThreadDetect, ThreadPrint, ThreadServer, ThreadUDP;
    int errThread;
    if(initApplication() == 0)
    {
        errThread = pthread_create(&ThreadDetect, NULL, &detectAndDisplay, NULL); //Create thread for managing new
client's requests
        if (errThread != 0)
        {
            printf("\nCan't create thread :[%s]", strerror(errThread));
        }
        errThread = pthread_create(&ThreadPrint, NULL, &print, NULL); //Create thread for managing new client's requests
        if (errThread != 0)
        {
            printf("\nCan't create thread :[%s]", strerror(errThread));
        }
        errThread = pthread_create(&ThreadServer, NULL, &runTcpServer, NULL); //Create thread for managing new
client's requests
        if (errThread != 0)
        {
            printf("\nCan't create thread :[%s]", strerror(errThread));
        }
        errThread = pthread_create(&ThreadUDP, NULL, &runUdpServer, NULL); //Create thread for managing new
client's requests
        if (errThread != 0)
        {
            printf("\nCan't create thread :[%s]", strerror(errThread));
        }
    }
    else
    {
        perror("Something went wrong when initializing the application!");
        return -1;
    }
    printf("Unknown command!\n=====\\n");

    fetchFrames(); //Thread-ul main se va ocupa de fetch frames
    //      while(1);

    cout<<"Program terminated!"<<endl;
    return 100;
}

```

frames.cpp

```
#include "frames.h"

using namespace std;
using namespace cv;

//=====
cv::CascadeClassifier face_cascade; // Obiect care administrează identificatorii Haar
long int start;
int height;
std::vector<cv::Mat> qFetch, qDetect;
std::vector<cv::Rect> faces; /* vector in care vor fi stocate fetele */
std::vector<cv::Mat> qFramesForStreaming;
bool streaming;
//=====
void* detectAndDisplay(void *argument)
{
    //          clock_t begin,end;
    /*
    -----Funcția de Detectare a fetelor-----
    */
}
```

```

Aceasta functie foloseste identificatori Haar pentru detectarea fetelor
*/
cout<<"Started detection!"<<endl;
Mat frame;
while(1)
{
    if(!qFetch.empty())
    {
        frame = qFetch.front(); /*In aceasta variabila va fi salvat ultimul frame adaugat in coada qFetch */
        qFetch.pop_back(); /* Ultimul frame adaugat se sterge din coada */

        //face_cascade.detectMultiScale(frame, faces, 1.2, 2, CASCADE_SCALE_IMAGE);

        //begin = clock(); /* Start procesare */
        face_cascade.detectMultiScale(frame, faces, 1.1, 3); /* detectia fetelor din imagine. */
        //end = clock(); /* Stop procesare - Nu stiu daca in acest interval nu intervine alt thread!! */
        //
        //double elapsed_secs = double(end - begin) / CLOCKS_PER_SEC; /* Calcul timp in secunde */
        //cout<<elapsed_secs<<" sec "<<endl;

        if(faces.size() > 0) /* Daca s-au detectat fete... */
        {
            for(unsigned int ic = 0 ; ic < faces.size() ; ic ++ )
            {
                Point *pt1 = new Point(faces[ic].x,faces[ic].y);
                Point *pt2 = new Point( (faces[ic].x + faces[ic].width) , (faces[ic].y +
faces[ic].height) );

                rectangle(frame,*pt1,*pt2,Scalar(255, 0, 0), 5, 8, 0);

                delete pt1;
                delete pt2;
                /* ...Acestea se parcurg si se incadreaza intr-un un dreptunghi */
            }

            if(qDetect.size() < limQDetect) /* Daca coada qDetect nu este plina */
            {
                qDetect.push_back(frame); /* Se adauga imaginea in coada si urmeaza sa fie afisata
de thread-ul de Print */

                faces.empty(); /* Se goleste vectorul fetelor pentru urmatorul frame */
            }
            else
            {
                qDetect.erase(qDetect.begin(),qDetect.begin()+qDetect.size()-1);
            }
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(30));
    }
}

}

//=====

void* print(void *argument)
{
    /*
    Functia de print preia o imagine din coada qDetect, daca exista, si o afiseaza pe ecran
    */
    Mat frame;
    cout<<"Started print!"<<endl;
    while(1)
    {
        if(!qDetect.empty() && !faces.empty()) /* Daca exista frame-uri in coada qDetect... */
        {
            frame = qDetect.front();
            qDetect.pop_back();
            imshow("Display window",frame); /* ...se afiseaza ultimul adaugat */
            waitKey(1);
        }
        else if(!qFetch.empty() && faces.empty())
        {
            /* TODO de vazut o modalitate de a afisa imagine de la camera fara ca o persoana sa
            * fie in cadru sau nu */

```

```

        frame = qFetch.front();
        qFetch.pop_back();

        imshow("Display window",frame); /* ...se afiseaza ultimul adaugat */
        waitKey(1);
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(30));
}

}

//=====

int initApplication()
{
    if(loadCascade(&face_cascade) == -1)
    {
        cerr<<"Error loading cascade_classifiers!"<<endl;
        return -2;
    }

    cv::setNumThreads(1);

    return 0;
}

//=====

void fetchFrames()
{
    //Functie rulata de thread-ul main, care preia imagini de la camera.
    Mat frame;
    //Mat frame_grayscale;
    cout<<"Started fetch!"<<endl;
    VideoCapture cap(0);
    if(!cap.isOpened())
    {
        cout<<"Could not open the webcam"<<endl; /* eroare la deschidere camera */
        exit(1); /* Incheie aplicatia cu eroare */

        /*
            Interesant de implementat spre final: Daca la deschiderea sistemului, apare o eroare,
            utilizatorul sa fie notificat, eventual sa poata da restart la aplicatie, remote.

            Am observat ca se mai intampla cateodata sa nu deschida camera, si atunci aplicatia
            se blocheaza.
        */
    }
    else{
        cout<<"Camera is opened"<<endl; /* Succes deschidere camera -> Aplicatia poate porni */
    }

    while(1)
    {
        if(cap.read(frame)) /* se citeste frame-ul */
        {
            start = cv::getTickCount(); /* salvez momentul curent de timp */
            if(qFetch.size() == limQFetch)
            {
                qFetch.erase(qFetch.begin(),qFetch.begin()+qFetch.size()-1);
            }
            qFetch.push_back(frame); /* Se adauga in coada de frame-uri neprocesate */

            if(streaming)
            {
                if(qFetch.size() == limQFetch)
                {
                    qFramesForStreaming.erase(qFramesForStreaming.begin(),qFramesForStreaming.begin()+qFramesForStreaming.size()-1);
                    qFramesForStreaming.push_back(frame);
                }
            }

            //std::this_thread::yield();
            std::this_thread::sleep_for(std::chrono::milliseconds(30)); /* thread-ul main cedeaza procesorul */
        }
    }
}

```

```

    }
}
//=====

```

tcpserver.cpp

```

#include "tcpserver.h"
//=====
int sockfdTcp, connfdTcp; // listen on sock_fd, new connection on new_fd
struct addrinfo hintsTcp, *servinfoTcp, *pTcp;
struct sockaddr_storage their_addrTcp; // connector's address information
socklen_t sin_size;
struct sigaction sa;
int yes=1;
char sTcp[INET6_ADDRSTRLEN];
int rvTcp;
int result;
bool streamIsRunning = false;
using namespace std;
std::vector<User> vectorUsers;
char logsPath[] = "/home/alex/Documents/logs.xml";
char usersPath[] = "/home/alex/Documents/users2.xml";
bool headerReceived = false, imageReceived = false;
//=====
void sendData(void* message, int client)
{
    if (send(client, message, strlen((char*)message), 0) == -1)
    {
        perror("send");
    }
}
//=====
char* receiveData(int client)
{
    char buffer[100000];
    int n1 = recv(client, buffer, 1024, 0);
    buffer[n1] = '\0';
    int nrChar = n1;
    if(strstr(buffer, MESSAGE_STOP_FLAG) == NULL && n1 > 0)
    {
        char aux[1025];
        while(strstr(buffer,MESSAGE_STOP_FLAG) == NULL)
        {
            int n = recv(client, aux, 1024,0);
            if(n > 0)
            {
                aux[n] = '\0';
                strncat(buffer,aux,strlen(aux)+1);
                nrChar += strlen(aux);
            }
        }
    }else if(n1 < 0)
    {
        perror("Error at receive data");
        return NULL;
    }else if(n1 == 0)
    {
        return NULL;
        printf("NULL\n");
    }

    char *message = 0;
    int nrCharNoFlag = strlen(buffer) - strlen(MESSAGE_STOP_FLAG) - 1;
    buffer[nrCharNoFlag] = '\0';
    message = (char*)malloc(nrCharNoFlag+1);
    strcpy(message, buffer);
    return message;
}
//=====
void* manageClient(void *argument)
{
    //Aceasta functie se ocupa de comunicatia cu clientul TCP de pe telefon.

    int connfdLocal = *(int*)argument;
    User lastAdmin;

```

```

while(1)
{
    char* buffer;
    buffer = receiveData(connfdLocal);

    if(buffer == NULL)
    {
        close(connfdLocal);
        break;
    }

    string sBuffer = buffer;
    string username, password;
    User mUser;
    free(buffer);

    size_t spatiu = sBuffer.find_first_of(' ');
    string tipSolicitare = sBuffer.substr(0,spatiu);
    sBuffer = sBuffer.substr(spatiu+1);

    if( (tipSolicitare.compare(ADMINISTRATOR_GET_USERS)!=0) &&
        (tipSolicitare.compare(ADMINISTRATOR_USERS_CHANGED)!=0) )
    {

        spatiu = sBuffer.find_first_of(' ');
        username = sBuffer.substr(0,spatiu);
        password = sBuffer.substr(spatiu+1);

        mUser = getUserWith(username,password);
    }

    char *message = 0;
    if(tipSolicitare.compare(LOGIN_USER_COMMAND) == 0)
    {
        if(mUser.getUsername().compare(username)==0 && mUser.getPassword().compare(password)==0 &&
            username.compare("")!=0 && password.compare("")!=0)
        {
            message = prepareMesaj((string)LOGIN_USER_RESPONSE_OK + " " +
                mUser.getFirstName() + " " + mUser.getLastName() + " " + bool2string(mUser.isAdministrator()));
            logAction(LOG_USER_LOGIN,mUser);
        }
        else
        {
            message = prepareMesaj(LOGIN_USER_RESPONSE_FAILED);
        }
    }
    else if(tipSolicitare.compare(HOME_OPEN_DOOR_BUTTON) == 0)
    {
        if(mUser.getPassword().compare(password) == 0)
        {
            message = prepareMesaj(HOME_OPEN_DOOR_BUTTON_RESPONSE_OK);
            logAction(LOG_ENTER,mUser);
        }
        else{
            message = prepareMesaj(HOME_OPEN_DOOR_BUTTON_RESPONSE_FAILED);
        }
    }
    else if(tipSolicitare.compare(HOME_OPEN_DOOR_FINGERPRINT) == 0)
    {
        message = prepareMesaj(HOME_OPEN_DOOR_FINGERPRINT_RESPONSE_OK);
        logAction(LOG_ENTER,mUser);
    }
    else if(tipSolicitare.compare(ADMINISTRATOR_LOGIN_COMMAND) == 0)
    {
        if(mUser.getUsername().compare(username)==0 && mUser.getPassword().compare(password)==0 &&
            mUser.isAdministrator())
        {
            message = prepareMesaj(ADMINISTRATOR_LOGIN_RESPONSE_OK);
            logAction(LOG_ADMIN_LOGIN,mUser);
            lastAdmin = mUser;
        }
        else{
            message = prepareMesaj(ADMINISTRATOR_LOGIN_RESPONSE_FAILED);
        }
    }
}

```

```

} else if (tipSolicitare.compare(ADMINISTRATOR_GET_USERS) == 0)
{
    char* fisierXML = readFileContent(usersPath);
    message = prepareMesaj(fisierXML);
    free(fisierXML);
}
else if (tipSolicitare.compare(ADMINISTRATOR_USERS_CHANGED) == 0)
{
    writeFileContent(usersPath, (char*)sBuffer.c_str());
    parseUsersXmlFile(usersPath, vectorUsers);
    // for(unsigned int i = 0; i < vectorUsers.size(); i++)
    // {
    //     vectorUsers.at(i).printUserInfo();
    // }
    logAction(LOG_ADMIN_USER_CHANGED, lastAdmin);
    continue;
}
else if (tipSolicitare.compare(HOME_START_STREAMING) == 0)
{
    logAction(LOG_STREAM_STARTED, mUser);
    streaming = true;
    // udpClients.push_back(connfdLocal);
    message = prepareMesaj(HOME_STREAM_STARTED);
}
else if (tipSolicitare.compare(HOME_STOP_STREAMING) == 0)
{
    logAction(LOG_STREAM_STOPPED, mUser);
    streaming = false;
    message = prepareMesaj(HOME_STREAM_STOPPED);
    qFramesForStreaming.erase(qFramesForStreaming.begin(), qFramesForStreaming.end());
}
else if (tipSolicitare.compare(HEADER_RECEIVED) == 0)
{
    headerReceived = true;
    continue;
}
else if (tipSolicitare.compare(IMAGE_RECEIVED) == 0)
{
    imageReceived = true;
    continue;
}
else if (tipSolicitare.compare(LOGS_LOGIN_COMMAND) == 0)
{
    if (mUser.getUsername().compare(username) == 0 && mUser.getPassword().compare(password) == 0 &&
mUser.isAdministrator())
    {
        message = prepareMesaj(LOGS_LOGIN_RESPONSE_OK);
        // logAction(LOG_ADMIN_LOGIN, mUser);
    } else {
        message = prepareMesaj(LOGS_LOGIN_RESPONSE_FAILED);
    }
}
else if (tipSolicitare.compare(LOGS_GET_LOG_LIST) == 0)
{
    char* fisierXML = readFileContent(logsPath);
    message = prepareMesaj(fisierXML);
    free(fisierXML);
} else {
    printf("Unknown command!\n\n");
    close(connfdLocal);
    message = prepareMesaj("");
    break;
}
sendData(message, connfdLocal);
free(message);
message = 0;
}
return NULL;
}
//=====
void* runTcpServer(void *argument)
{
    //Se asteapta conexiuni noi, pentru care se creeaza thread separat
    int errThread, errServer;

```

```

pthread_t tid;
parseUsersXmlFile(usersPath, vectorUsers);
errServer = initTCPServer();

if(errServer != 0)
{
    printf("There was an error when initializing the server! Error code: %d \n",result);
    return NULL;
}

parseLogsXmlFile(logsPath);

while(1)
{
    printf("server: waiting for connections...\n");

    sin_size = sizeof their_addrTcp;

    connfdTcp = accept(sockfdTcp, (struct sockaddr *)&their_addrTcp, &sin_size);
    if(connfdTcp < 0)
    {
        printf("Error in accept\n");
        continue;
    }

    inet_ntop(their_addrTcp.ss_family,      get_in_addr((struct sockaddr *)&their_addrTcp),sTcp, sizeof sTcp);
    printf("server: got connection from %s\n", sTcp);

    errThread = pthread_create(&tid, NULL, &manageClient, &connfdTcp); //Create thread for managing new client's
requests
    if (errThread != 0)
    {
        printf("\nCan't create thread :[%s]", strerror(errThread));
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(30));
}

return NULL;
}
//=====

```

udpserver.cpp

```

#include "udpserver.h"

int sockfdUdp;          /* socket */
unsigned int clientlen; /* byte size of client's address */
struct sockaddr_in serveraddr; /* server's addr */
struct sockaddr_in clientaddr; /* client addr */
struct hostent *hostp; /* client host info */
char bufRecvUdp[BUFSIZE] = "msg"; /* message buf */
char *bufSendUdp = new char[BUFSIZE + 1];
char *hostaddrp; /* dotted decimal host addr string */
int broadcastEnable = 1; /* flag value for setsockopt */
int n; /* message byte size */

```

```

std::vector<sockaddr_in> clients;

//=====================================================
void sendToUdpClient(uchar* buf, int length)
{
    n = sendto(sockfdUdp, buf, length, 0, (struct sockaddr *) &clientaddr, clientlen);

    if (n < 0)
    {
        printf("ERROR in sendto\n");
    }
    else{
        //                printf("\nSent %d bytes\n ",n);
    }
}
//=====================================================
char* receiveFromUdpClient()
{
    strcpy(bufRecvUdp, "");
    n = recvfrom(sockfdUdp, bufRecvUdp, BUFSIZE, 0, (struct sockaddr *) &clientaddr, &clientlen);
    if (n < 0)
    {
        printf("ERROR in recvfrom , %s", bufRecvUdp);
    }
    printf("server received %d/%d bytes: %s\n", (int)strlen(bufRecvUdp), n, bufRecvUdp);
    return bufRecvUdp;
}
//=====================================================
void* runUdpServer(void *argument)
{
    if(initUDPServer() != 0)
    {
        printf("Error at initializing the UDP server\n");
    }

    uchar* imaginePNG = 0;
    cv::Mat currentFrame;
    std::vector<uchar> buf;
    int begin, end;

    while(strcmp(receiveFromUdpClient(), HOME_START_STREAMING) != 0) {}

    while (1)
    {
        if(streaming && !qFramesForStreaming.empty())
        {
            begin = 0;
            end = 0;
            currentFrame = qFramesForStreaming.back();
            qFramesForStreaming.pop_back();
            //                cv::Mat grayFrame;
            //                cv::cvtColor(currentFrame, grayFrame, CV_BGR2GRAY);
            buf.erase(buf.begin(), buf.end());
            cv::imencode(".jpg", currentFrame, buf);
            printf("NrBytes: %d\n", (int)buf.size());
            buf.shrink_to_fit();

            unsigned int numarPachete, numarBytes;
            numarBytes = buf.size();

            if(numarBytes % (BUFSIZE - sizeof(unsigned int)) != 0)
            {
                numarPachete = numarBytes / (BUFSIZE - sizeof(unsigned int)) + 1;
            }
            else
            {
                numarPachete = numarBytes / (BUFSIZE - sizeof(unsigned int));
            }

            std::vector<uchar> header;
            for(int i = sizeof(unsigned int) - 1 ; i >= 0 ; i--)
            {
                header.insert(header.begin(), GET_BYTE(numarBytes, i, sizeof(unsigned int)));
            }
            //adaug numarul total de bytes din imagine
            for(int i = sizeof(unsigned int)-1 ; i >= 0 ; i--)

```



```

{
    header.insert(header.begin(), GET_BYTE(numarPachete,i,sizeof(unsigned int)));
} //adaug numarul total de pachete

header.shrink_to_fit();

imagePNG = header.data();
sendToUdpClient(imagePNG, header.size()); //Trimit cele doua valori clientului

while(!headerReceived && streaming) //verific daca clientul a primit valorile
{
    std::this_thread::sleep_for(std::chrono::milliseconds(5));
}

headerReceived = false;

for(unsigned int k = 0 ; k < numarPachete ; k++) //trimit pachete de cate BUFSIZE octeti
{
    if(k == numarPachete - 1)
    {
        end = buf.size() - 1;
        std::vector<uchar> newBuf(&buf[begin],&buf[end]);
        for(int i = sizeof(unsigned int)-1 ; i >= 0 ; i--)
        {
            newBuf.insert(newBuf.begin() , GET_BYTE(k,i,sizeof(unsigned int)));
        }
        printf("\n=====\\n");
        printf("Index: %d\\n",k);
        printf("Begin: %d\\n",begin);
        printf("End: %d\\n",end);
        printf("Total: %d\\n",(int)buf.size());
        printf("End - Begin: %d\\n", end - begin + 1);
        newBuf.shrink_to_fit();
        imagePNG = newBuf.data();
        printf("Length: %d\\n",(int)newBuf.size());
        sendToUdpClient(imagePNG,newBuf.size());
        printf("=====\\n");

        while(!imageReceived && streaming)
        {
            std::this_thread::sleep_for(std::chrono::milliseconds(5));
        }

        imageReceived = false;
        //daca clientul a raspuns, inseamna ca se poate trece la urmatorul frame
    }else
    {
        printf("\n=====\\n");
        printf("Index: %d\\n",k);

        end = end + (BUFSIZE - sizeof(unsigned int));
        printf("Begin: %d\\n",begin);
        printf("End: %d\\n",end);
        printf("Total: %d\\n",(int)buf.size());
        printf("End - Begin: %d\\n", end - begin);

        std::vector<uchar> newBuf(&buf[begin],&buf[end]);
        for(int i = sizeof(unsigned int)-1 ; i >= 0 ; i--)
        {
            newBuf.insert(newBuf.begin() , GET_BYTE(k,i,sizeof(unsigned int)));
        }
        newBuf.shrink_to_fit();
        imagePNG = newBuf.data();
        printf("Length: %d\\n",(int)newBuf.size());
        sendToUdpClient(imagePNG,newBuf.size());
        std::this_thread::sleep_for(std::chrono::milliseconds(10));

        begin = end;
        //
        end = end + 1;
        printf("=====\\n");
    }
}
}

```

```

        }
        std::this_thread::sleep_for(std::chrono::milliseconds(5));
    }
    printf("Closing UDP server...\n");
    return NULL;
}

```

User.h

```

class User {
//=====
private:
    std::string mFirstName;
    std::string mLastName;
    std::string mUsername;
    std::string mPassword;
    bool mIsAdministrator;
//=====
public:
    User(std::string firstName, std::string lastName, std::string username, std::string password, bool administrator);
    User();
//=====
    void setFirstName(std::string firstName);
    std::string getFirstName();
//=====
    void setLastName(std::string lastName);
    std::string getLastName();
//=====
    void setUsername(std::string username);
    std::string getUsername();
//=====
    void setPassword(std::string password);
    std::string getPassword();
//=====
    void setIsAdministrator(bool isAdministrator);
    bool isAdministrator();
//=====
    void printUserInfo();
//=====
    ~User();
};

```

LogAction.h

```

class LogAction {
//=====
private:
    std::string mActionType;
    std::string mTime;
    User mUser;
//=====
public:
    LogAction();
//=====
    std::string getActionType();
    void setActionType(std::string actionType);
//=====
    std::string getTime();
    void setTime(std::string time);
//=====
    User getUser();
    void setUser(User user);
//=====
    virtual ~LogAction();
};

```

Anexa 2 – Cod sursă aplicația mobilă

HomeFragment.java

```
public class HomeFragment extends Fragment
    implements TcpObserver,UdpObserver{

    public static final String TAG = "HOME_FRAGMENT";
    AlertDialog dialog;
    private TextView doorStatus;
    FingerprintChecker mFingerprintChecker;
    private Button btnUnlock, streamStartButton;
    ImageView imageView;
    private static User mUser;
    boolean streaming = false;

    static{
        System.loadLibrary("opencv_java3");
    }
    //=====

    void openDoorAndCloseAfter(int miliseconds){
        doorStatus.setText("UNLOCKED");
        doorStatus.setTextColor(Color.GREEN);

        new CountDownTimer(miliseconds, 1000) {
            public void onTick(long millisUntilFinished) {
            }
            public void onFinish() {
                doorStatus.setText("LOCKED");
                doorStatus.setTextColor(Color.RED);
            }
        }.start();
    } //=====

    public View onCreateView(LayoutInflater inflater, @Nullable ViewGroup container,
        @Nullable Bundle savedInstanceState) {
        return inflater.inflate(R.layout.home_fragment_layout, null, false);
    }
    //=====

    public void onViewCreated(View view, @Nullable Bundle savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);

        getActivity().setTitle("Home");
        doorStatus = (TextView) view.findViewById(R.id.etDoorStatus);
        btnUnlock = (Button) view.findViewById(R.id.btnUnlock);
        streamStartButton = (Button) view.findViewById(R.id.btnStartStream);
        TcpClient.getInstance().Attach(this);
        UdpClient.getInstance().Attach(this);
        imageView = (ImageView) view.findViewById(R.id.framesImageView);
        mUser = MainActivity.getCurrentUser();

        btnUnlock.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                dialog = null;
                showLoginDialog();
                //Checking fingerprint hardware.....
                mFingerprintChecker.testFingerprintHardwareAndStartAuthetification(dialog);
            }
        });
        streamStartButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                if(!streaming) {
                    streaming = true;
                    TcpClient.getInstance().sendMessageToServer(Constants.HOME_START_STREAMING);
                    UdpClient.getInstance().sendToUdp(Constants.HOME_START_STREAMING);
                    streamStartButton.setText("STOP STREAM");
                    streamStartButton.setTextColor(Color.RED);
                }else if(streaming){
                    streaming = false;
                }
            }
        });
    }
}
```

```

        TcpClient.getInstance().sendMessageToServer(Constants.HOME_STOP_STREAMING);
        UdpClient.getInstance().sendToUdp(Constants.HOME_STOP_STREAMING);
        streamStartButton.setText("START STREAM");
        streamStartButton.setTextColor(Color.BLACK);
    }
}
});
}

@Override
public void onStop() {
    super.onStop();
    TcpClient.getInstance().Detach(this);
}

@Override
public void updateTcp(String message) {

    if(message.equals(Constants.HOME_OPEN_DOOR_BUTTON_RESPONSE_OK) ||
        message.equals(Constants.HOME_OPEN_DOOR_FINGERPRINT_RESPONSE_OK)){
        dialog.dismiss();
        dialog = null;
        openDoorAndCloseAfter(10000);

    }else if(message.equals(Constants.HOME_OPEN_DOOR_BUTTON_RESPONSE_FAILED)){
        Toast.makeText(getActivity(), "Wrong password!", Toast.LENGTH_LONG);
    }
}

@Override
public void updateUdp(final byte[] image) {
    final Bitmap bmp = BitmapFactory.decodeByteArray(image, 0, image.length);
    // imageView.setImageBitmap(Bitmap.createScaledBitmap(bmp, 640, 480, false));
    // Bitmap bmp = arrayToBitmap(image, 640, 480);

    Utils.runOnUiThread(new Runnable() {
        @Override
        public void run() {
            imageView.setImageBitmap(bmp);
        }
    });
}

//=====
void showLoginDialog() {
    AlertDialog.Builder mBuilder = new AlertDialog.Builder(getActivity());
    View mView = getLayoutInflater().inflate(R.layout.dialog_login, null);

    doorStatus.setTextColor(Color.RED);
    final EditText mPassword = (EditText) mView.findViewById(R.id.etDialogPassword);
    final Button btnLogin = (Button) mView.findViewById(R.id.btnLogin);

    mBuilder.setView(mView);
    dialog = mBuilder.create();

    EditText etUsername = (EditText) mView.findViewById(R.id.etDialogUsername) ;
    etUsername.setVisibility(View.GONE);

    mFingerprintChecker = new FingerprintChecker();

    btnLogin.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View view) {
            if (mPassword.getText().toString() != null) {
                TcpClient.getInstance().sendMessageToServer(Constants.HOME_OPEN_DOOR_BUTTON + " " + mUser.getUsername() + " " +
mPassword.getText().toString());
                //TODO: de trimis si username-ul odata cu parola introdusa de utilizator in UI (username din memorie)
            }
        }
    });
    dialog.show();
}

```

AdministratorFragment.java

```

public class AdministratorFragment extends Fragment
    implements TcpObserver{

    public static final String TAG = "ADMIN_FRAGMENT";
    ListView usersListView;
    ArrayList<User> usersList;
    private static UsersCustomAdapter adapter;
    String mXmlContent;
    //=====
    public View onCreateView(LayoutInflater inflater, @Nullable ViewGroup container, @Nullable Bundle savedInstanceState) {
        return inflater.inflate(R.layout.administrator_layout, null, false);
    } //=====
    public void onViewCreated(View view, @Nullable Bundle savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);
        setHasOptionsMenu(true);

        getActivity().setTitle(" Administrator");
        usersListView = (ListView)view.findViewById(R.id.usersList);
        usersList = new ArrayList<User>();
        mXmlContent = new String();
        TcpClient.getInstance().Attach(this);
        TcpClient.getInstance().sendMessageToServer(Constants.ADMINISTRATOR_GET_USERS);

        adapter = new UsersCustomAdapter(usersList, getActivity(), new UsersCustomAdapter.OnEditButtonClicked() {
            @Override
            public void onEditButtonClicked(User user, int position) {
                showEditDialog(user,position);
            }
        });
        usersListView.setAdapter(adapter);

        usersListView.setOnItemLongClickListener(new AdapterView.OnItemLongClickListener() {
            @Override
            public boolean onItemLongClick(AdapterView<?> adapterView, View view, int i, long l) {
                showYesNoDialog(i, view);
                return true;
            }
        });
    }
    //=====
    public void onCreateOptionsMenu(Menu menu, MenuInflater inflater) {
        inflater.inflate(R.menu.administrator_menu,menu);
        super.onCreateOptionsMenu(menu, inflater);
    }
    //=====
    public boolean onOptionsItemSelected(MenuItem item) {
        switch (item.getItemId())
        {
            case R.id.add_user:
                showAddDialog();
                return true;
        }

        return super.onOptionsItemSelected(item);
    }
    //=====
    void showEditDialog(final User user, final int position){
        final AlertDialog dialog;
        AlertDialog.Builder mBuilder = new AlertDialog.Builder(getActivity());
        final View dialogView = getLayoutInflater().inflate(R.layout.edit_add_user_dialog_layout, null);
        mBuilder.setView(dialogView);
        dialog = mBuilder.create();

        final EditText etFirstName, etLastName, etUsername, etPassword;
        final CheckBox cbIsAdministrator;
        final Button btnCancelEdit, btnOkEdit;

        etFirstName = (EditText)dialogView.findViewById(R.id.etFirstName);
        etLastName = (EditText)dialogView.findViewById(R.id.etLastName);
        etUsername = (EditText)dialogView.findViewById(R.id.etUsername);
        etPassword = (EditText)dialogView.findViewById(R.id.etPassword);
        cbIsAdministrator = (CheckBox)dialogView.findViewById(R.id.cbIsAdministrator);
        btnCancelEdit = (Button)dialogView.findViewById(R.id.btnCancelEdit);
        btnOkEdit = (Button)dialogView.findViewById(R.id.btnOkEdit);
    }

```

```

etFirstName.setText(user.getFirstName());
etLastName.setText(user.getLastName());
etUsername.setText(user.getUsername());
etPassword.setText(user.getPassword());
cbIsAdministrator.setChecked(user.isAdministrator());

btnCancelEdit.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        dialog.dismiss();
    }
});

btnOkEdit.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        if(!etFirstName.getText().toString().matches("") &&
            !etLastName.getText().toString().matches("") &&
            !etUsername.getText().toString().matches("") &&
            !etPassword.getText().toString().matches("")) {
            usersList.get(position).setFirstName(etFirstName.getText().toString());
            usersList.get(position).setLastName(etLastName.getText().toString());
            usersList.get(position).setUsername(etUsername.getText().toString());
            usersList.get(position).setPassword(etPassword.getText().toString());
            usersList.get(position).setIsAdministrator(cbIsAdministrator.isChecked());

            adapter.editUserAt(usersList.get(position), position);

            Toast.makeText(view.getContext(), "User " + etFirstName.getText().toString() + " " +
                etLastName.getText().toString() + " edited successfully", Toast.LENGTH_LONG).show();
            dialog.dismiss();
            new XmlHandler().writeUsersXml();
            TcpClient.getInstance().sendMessageToServer(Constants.ADMINISTRATOR_USERS_CHANGED + " " + mXmlContent);

        } else
        {
            Toast.makeText(view.getContext(), "Missing information", Toast.LENGTH_LONG).show();
        }
    }
});
dialog.show();
}
@Override
public void onStop() {
    super.onStop();
    TcpClient.getInstance().Detach(this);
}
//=====================================================
void showAddDialog(){
    final AlertDialog dialog;
    AlertDialog.Builder mBuilder = new AlertDialog.Builder(getActivity());
    final View dialogView = getLayoutInflater().inflate(R.layout.edit_add_user_dialog_layout, null);
    mBuilder.setView(dialogView);
    dialog = mBuilder.create();

    final EditText etFirstName, etLastName, etUsername, etPassword;
    final CheckBox cbIsAdministrator;
    final Button btnCancelEdit, btnOkEdit;

    etFirstName = (EditText)dialogView.findViewById(R.id.etFirstName);
    etLastName = (EditText)dialogView.findViewById(R.id.etLastName);
    etUsername = (EditText)dialogView.findViewById(R.id.etUsername);
    etPassword = (EditText)dialogView.findViewById(R.id.etPassword);
    cbIsAdministrator = (CheckBox)dialogView.findViewById(R.id.cbIsAdministrator);
    btnCancelEdit = (Button)dialogView.findViewById(R.id.btnCancelEdit);
    btnOkEdit = (Button)dialogView.findViewById(R.id.btnOkEdit);

    btnCancelEdit.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View view) {
            dialog.dismiss();
        }
    });

    btnOkEdit.setOnClickListener(new View.OnClickListener() {

```

```

@Override
public void onClick(View view) {

    User user = new User();

    if(!etFirstName.getText().toString().matches("") &&
        !etLastName.getText().toString().matches("") &&
        !etUsername.getText().toString().matches("") &&
        !etPassword.getText().toString().matches("")) {

        //TcpClient.getInstance().sendMessageToServer(Constants.ADMINISTRATOR_USERS_CHANGED);

        user.setFirstName(etFirstName.getText().toString());
        user.setLastName(etLastName.getText().toString());
        user.setUsername(etUsername.getText().toString());
        user.setPassword(etPassword.getText().toString());
        user.setIsAdministrator(cbIsAdministrator.isChecked());

        usersList.add(user);
        adapter.notifyDataSetChanged();
        // Toast.makeText(getActivity(), "User " + etFirstName.getText().toString()+" "+
        // etLastName.getText().toString()+" added successfully",Toast.LENGTH_LONG).show();
        dialog.dismiss();
        new XmlHandler().writeUsersXml();
        TcpClient.getInstance().sendMessageToServer(Constants.ADMINISTRATOR_USERS_CHANGED + " " + mXmlContent);
    }else
    {
        Toast.makeText(view.getContext(), "Missing information", Toast.LENGTH_LONG).show();
    }
}
});
dialog.show();
} //=====
void showYesNoDialog(final int position, final View view){
    AlertDialog.Builder builder = new AlertDialog.Builder(getActivity());

    builder.setTitle("Confirm");
    builder.setMessage("Are you sure you want to delete user " + "\"" + usersList.get(position).getFirstName()
        + " " + usersList.get(position).getLastName()+ "\"" + "?");

    builder.setPositiveButton("YES", new DialogInterface.OnClickListener() {

        public void onClick(DialogInterface dialog, int which) {
            Snackbar.make(view, "User " + "\"" + usersList.get(position).getFirstName()
                + " " + usersList.get(position).getLastName()+ "\"" + " deleted!",Snackbar.LENGTH_SHORT).show();
            usersList.remove(position);
            adapter.notifyDataSetChanged();
            dialog.dismiss();
            new XmlHandler().writeUsersXml();
            TcpClient.getInstance().sendMessageToServer(Constants.ADMINISTRATOR_USERS_CHANGED + " " + mXmlContent);
        }
    });

    builder.setNegativeButton("NO", new DialogInterface.OnClickListener() {

        @Override
        public void onClick(DialogInterface dialog, int which) {
            dialog.dismiss();
        }
    });

    AlertDialog alert = builder.create();
    alert.show();
}
//=====
@Override
public void updateTcp(String message) {
    if(message.contains(Constants.XML_VERSION)){
        new XmlHandler().parseUsersXml(message);
    }
}
//=====
private class XmlHandler{
    private String XML_VERSION = "<?xml version='1.0' encoding='UTF-8'?">",
        ROOT_NODE_START = "<users_document>",

```

```

        ROOT_NODE_END = "</users_document>",
        USER_NODE_START = "<user>",
        USER_NODE_END = "</user>",
        FIRST_NAME_START = "<first_name>",
        FIRST_NAME_END = "</first_name>",
        LAST_NAME_START = "<last_name>",
        LAST_NAME_END = "</last_name>",
        USERNAME_START = "<username>",
        USERNAME_END = "</username>",
        PASSWORD_START = "<password>",
        PASSWORD_END = "</password>",
        ADMINISTRATOR_START = "<administrator>",
        ADMINISTRATOR_END = "</administrator>";
//=====
void parseUsersXml(String usersXml){
    try {
        InputStream is = new ByteArrayInputStream(usersXml.getBytes("UTF-8"));

        DocumentBuilderFactory dbFactory = DocumentBuilderFactory.newInstance();
        DocumentBuilder dBuilder = dbFactory.newDocumentBuilder();
        Document doc = dBuilder.parse(is);

        Element element=doc.getDocumentElement();
        element.normalize();

        NodeList nList = doc.getElementsByTagName("user");

        for (int i=0; i<nList.getLength(); i++) {

            Node node = nList.item(i);

            if (node.getNodeType() == Node.ELEMENT_NODE) {
                Element element2 = (Element) node;
                User user = new User();
                user.setFirstName(getValue("first_name", element2));
                user.setLastName(getValue("last_name", element2));
                user.setUsername(getValue("username", element2));
                user.setPassword(getValue("password", element2));
                user.setIsAdministrator(string2Bool(getValue("administrator", element2)));
                usersList.add(user);
                adapter.notifyDataSetChanged();
            }
        }

    } catch (Exception e) {
        e.printStackTrace();
    }
}
//=====
boolean string2Bool(String m){
    if(m.equals("true")){
        return true;
    }else{
        return false;
    }
}
//=====
String bool2string(boolean b){
    if(b)
        return "true";
    else
        return "false";
}
//=====
void writeUsersXml(){
    mXmlContent = "";
    mXmlContent = mXmlContent.concat(XML_VERSION + "\n" + ROOT_NODE_START + "\n");
    for(int i = 0 ; i < usersList.size() ; i++){
        User user = usersList.get(i);
        mXmlContent = mXmlContent.concat("\t"+USER_NODE_START + "\n"+
            "\t\t"+FIRST_NAME_START + user.getFirstName() + FIRST_NAME_END+"\n"+
            "\t\t"+LAST_NAME_START + user.getLastName() + LAST_NAME_END+"\n"+
            "\t\t"+USERNAME_START + user.getUsername() + USERNAME_END+"\n"+
            "\t\t"+PASSWORD_START + user.getPassword() + PASSWORD_END+"\n"+

```



```

                "\\t\\t"+ADMINISTRATOR_START + bool2string(user.isAdministrator()) + ADMINISTRATOR_END+"\\n"+
                "\\t"+USER_NODE_END + "\\n");
        }
        mXmlContent = mXmlContent.concat(ROOT_NODE_END);
    }
    private String getValue(String tag, Element element) {
        NodeList nodeList = element.getElementsByTagName(tag).item(0).getChildNodes();
        Node node = nodeList.item(0);
        return node.getNodeValue();
    }
}
}
}
//=====
public class UsersCustomAdapter extends ArrayAdapter<User> implements View.OnClickListener{

    private ArrayList<User> usersList;
    Context mContext;
    private int lastPosition = -1;
    OnEditButtonClicked meditButtonClickedListener;

    public UsersCustomAdapter(ArrayList<User> data, Context context, OnEditButtonClicked listener) {
        super(context, R.layout.users_row_item, data);
        this.usersList = data;
        this.mContext=context;
        meditButtonClickedListener = listener;
    }

    public View getView(int position, @Nullable View convertView, @NonNull ViewGroup parent) {

        User user = getItem(position);
        ViewHolder viewHolder;

        final View result;

        if(convertView == null) {

            viewHolder = new ViewHolder();
            LayoutInflater inflater = LayoutInflater.from(getContext());
            convertView = inflater.inflate(R.layout.users_row_item, parent, false);
            viewHolder.txtFirstName = (TextView)convertView.findViewById(R.id.userFirstName);
            viewHolder.txtLastName = (TextView)convertView.findViewById(R.id.userLastName);
            viewHolder.txtUsername = (TextView)convertView.findViewById(R.id.userUsername);
            viewHolder.txtIsAdministrator = (TextView)convertView.findViewById(R.id.userIsAdministrator);
            viewHolder.editUser = (ImageView)convertView.findViewById(R.id.ivEditUser);

            result = convertView;
            convertView.setTag(viewHolder);
        }else{
            viewHolder = (ViewHolder)convertView.getTag();
            result = convertView;
        }

        Animation animation = AnimationUtils.loadAnimation(mContext, (position > lastPosition) ? R.anim.up_from_bottom :
R.anim.down_from_top);
        result.startAnimation(animation);

        viewHolder.txtFirstName.setText(user.getFirstName());
        viewHolder.txtLastName.setText(user.getLastName());
        viewHolder.txtUsername.setText(user.getUsername());

        if(user.isAdministrator())
        {
            viewHolder.txtIsAdministrator.setText("admin");
        }else{
            viewHolder.txtIsAdministrator.setText("user");
        }

        viewHolder.editUser.setOnClickListener(this);
        viewHolder.editUser.setTag(position);

        return convertView;
    }
}

```

```

private static class ViewHolder {
    TextView txtFirstName;
    TextView txtLastName;
    TextView txtUsername;
    TextView txtIsAdministrator;
    ImageView editUser;
}

@Override
public void onClick(View view) {
    int position = (Integer)view.getTag();
    Object object = getItem(position);
    User selectedUser = (User)object;

    switch (view.getId())
    {
        case R.id.ivEditUser:

            meditButtonClickedListener.onEditButtonClicked(selectedUser,position);
    }
}

public interface OnEditButtonClicked{
    public void onEditButtonClicked(User user, int position);
}

void editUserAt(User user, int position){
    usersList.set(position,user);
    notifyDataSetChanged();
}

void addNewUser(User user){
    usersList.add(user);
    notifyDataSetChanged();
}
}
//=====
public class LogsCustomAdapter extends ArrayAdapter<LogAction> {

    private ArrayList<LogAction> logsList;
    Context mContext;
    private int lastPosition = -1;

    public LogsCustomAdapter(ArrayList<LogAction> data, Context context) {
        super(context, R.layout.log_row_item, data);
        this.logsList = data;
        this.mContext = context;
    }

    public View getView(int position, @Nullable View convertView, @NonNull ViewGroup parent) {

        LogAction act = getItem(position);
        User user = act.getLogUser();
        ViewHolder viewHolder;

        final View result;

        if(convertView == null) {

            viewHolder = new ViewHolder();
            LayoutInflater inflater = LayoutInflater.from(getContext());
            convertView = inflater.inflate(R.layout.log_row_item, parent, false);
            viewHolder.txtActionType = (TextView)convertView.findViewById(R.id.logActionType);
            viewHolder.txtUserName = (TextView)convertView.findViewById(R.id.logUserName);
            viewHolder.txtLogTime = (TextView)convertView.findViewById(R.id.logTime);
            viewHolder.txtLogDate = (TextView)convertView.findViewById(R.id.logDate);

            result = convertView;
            convertView.setTag(viewHolder);
        }else{
            viewHolder = (ViewHolder)convertView.getTag();
            result = convertView;
        }
    }
}

```

```

        Animation animation = AnimationUtils.loadAnimation(mContext, (position > lastPosition) ? R.anim.up_from_bottom :
R.anim.down_from_top);
        result.startAnimation(animation);

        viewHolder.txtActionType.setText(act.getLogType());
        viewHolder.txtUserName.setText(user.getFirstName() + "\n" + user.getLastName());
        String logTime = act.getLogTime();
        String logDate = act.getLogDayName() + " " + act.getLogDayNumber() + " " + act.getLogMonth() + " " + act.getLogYear();

        viewHolder.txtLogTime.setText(logTime);
        viewHolder.txtLogDate.setText(logDate);

        return convertView;
    }
    public void changeData(ArrayList<LogAction> newData)
    {
        logsList.clear();
        logsList = newData;
    }

    private static class ViewHolder {
        TextView txtActionType;
        TextView txtUserName;
        TextView txtLogTime;
        TextView txtLogDate;
    }
}

```